

Implémentation et évaluation de StreamKM++ sur Apache Spark

Jules MAURICE
Asma BEN-ZINE
Victorien GBAGUIDI

Août 2026

Table des matières

1	Introduction	3
2	Algorithmes	4
2.1	Algorithme de Lloyd pondéré	4
2.2	Initialisation k-means++	5
2.3	Construction du coreset	5
2.4	Merge-and-reduce	7
3	Apache Spark	8
3.1	RDD et DataFrames	8
3.2	Traitement par micro-batches avec Structured Streaming	8
3.3	Diffusion des centres avec broadcast	9
3.4	Réduction arborescente	9
3.5	Partitions et ressources de calcul	10
4	Implémentation	11
4.1	StreamKM++ distribué, flux borné	11
4.1.1	Coresets partiels par partition	11
4.1.2	Fusion arborescente du coreset final	11
4.1.3	Boucle Structured Streaming complète	12
4.1.4	Table de correspondance	12
4.2	Méthodologie d'évaluation du coût	12
4.3	Variante à requêtes périodiques (décrite, non implémentée)	13
4.4	Ce qui se traduit depuis Flink, ce qui ne se traduit pas	14
5	Évaluation expérimentale	15
5.1	Protocole expérimental	15
5.2	Validation de l'implémentation	16
5.3	Qualité du clustering sur Coverttype	16
5.3.1	Influence du nombre de clusters	17
5.3.2	Comparaison avec Spark MLlib	17
5.3.3	Influence de la taille du coreset	18
5.3.4	Limites de l'évaluation de qualité	18
5.4	Scalabilité de l'implémentation Spark	19
5.5	Optimisation du noyau local	19
5.5.1	Hypothèse	19
5.5.2	Protocole JMH	20
5.5.3	Résultats	20
5.6	Synthèse	20

6	Discussion	22
6.1	Apports de l'implémentation	22
6.2	Limites de l'implémentation	23
6.3	Bilan	25
7	Conclusion et perspectives	26
7.1	Travaux futurs	26
A	Code source complet	27
A.0.1	Configuration du projet	27
A.0.2	Couche core : structures de données et k-means	28
A.0.3	Couche coreset : coreset tree et merge-and-reduce	34
A.0.4	Couche spark : portage distribué	42
A.0.5	Couche experiments : harnais de mesure de qualité	46
A.0.6	Fonction Main	63
A.0.7	Suite de tests	64

Chapitre 1

Introduction

Le clustering de grands volumes de données pose des contraintes particulières lorsque les observations arrivent sous forme de flux. Contrairement à une approche batch classique, il n'est pas toujours possible de conserver l'ensemble des données en mémoire ou de les parcourir plusieurs fois. Les algorithmes de clustering destinés aux flux doivent donc limiter leur consommation mémoire tout en maintenant une qualité de partitionnement satisfaisante.

StreamKM++, proposé par Ackermann et al. (2012), répond à cette problématique en construisant progressivement un *coreset*, c'est-à-dire un résumé pondéré et de taille limitée des données observées. Le clustering final est ensuite calculé sur ce résumé à l'aide de k-means++, ce qui permet de réduire fortement la quantité de données manipulée lors de l'étape de clustering.

Bitsakis (2018) a proposé une implémentation distribuée de StreamKM++ avec Apache Flink. Le présent projet étudie le portage de cette approche vers **Apache Spark**, en particulier au moyen des RDD et de Structured Streaming. L'objectif n'est pas uniquement de traduire l'implémentation existante, mais d'adapter le traitement aux mécanismes propres à Spark, notamment le partitionnement des données, les opérations de réduction arborescente et l'exécution par micro-batches.

Le travail est organisé autour de trois objectifs. Le premier consiste à implémenter et valider les composants algorithmiques nécessaires à StreamKM++ : k-means pondéré, initialisation k-means++, construction du coreset et stratégie *merge-and-reduce*. Le deuxième porte sur leur distribution avec Spark, chaque partition construisant un résumé local avant la fusion des résultats. Enfin, l'implémentation est évaluée expérimentalement afin d'étudier la qualité du clustering obtenu, l'influence de la taille du coreset et le comportement de la solution lorsque le volume de données et le parallélisme augmentent.

L'architecture retenue sépare la logique algorithmique de l'orchestration distribuée. Les packages `streamkm.core` et `streamkm.coreset` contiennent les composants indépendants de Spark, tandis que `streamkm.spark` assure leur exécution distribuée. Cette organisation permet de vérifier séparément la correction de l'algorithme avant d'évaluer son comportement dans un environnement Spark.

Chapitre 2

Algorithmes

StreamKM++ est un algorithme de clustering conçu pour traiter des flux de données dont la taille peut être trop importante pour permettre la conservation de l'ensemble des observations en mémoire. Son principe consiste à maintenir progressivement un *coreset*, c'est-à-dire un résumé pondéré et de taille limitée du flux déjà observé. Le clustering final est ensuite calculé sur ce résumé plutôt que sur l'ensemble complet des données.

L'approche repose sur quatre composants principaux : une version pondérée de l'algorithme de Lloyd, l'initialisation k-means++, la construction d'un coreset à l'aide d'un arbre de partitionnement, puis une stratégie *merge-and-reduce* permettant de maintenir ce résumé au cours du flux. Ces composants sont décrits dans les sections suivantes, en distinguant la formulation algorithmique issue de la littérature des choix effectués dans notre implémentation lorsque cela est nécessaire.

2.1 Algorithme de Lloyd pondéré

L'algorithme de Lloyd constitue l'étape d'optimisation utilisée par k-means. À partir d'un ensemble de centres initiaux, il alterne deux opérations jusqu'à convergence : chaque point est d'abord affecté au centre le plus proche, puis chaque centre est recalculé comme la moyenne des points qui lui sont associés.

Dans StreamKM++, cette opération est réalisée sur un coreset et non directement sur les observations originales. Les représentants du coreset possèdent donc un poids indiquant la masse de données qu'ils représentent. L'algorithme de Lloyd doit par conséquent prendre ces poids en compte lors du recalcul des centres.

Soit un ensemble de points pondérés

$$P = \{(p_i, w_i)\}_{i=1}^n$$

et un ensemble de centres

$$C = \{c_1, \dots, c_k\}.$$

Après l'affectation de chaque point à son centre le plus proche, le centre du cluster j est recalculé selon :

$$c_j = \frac{\sum_{p_i \in C_j} w_i p_i}{\sum_{p_i \in C_j} w_i}.$$

La qualité du clustering est évaluée à l'aide de la somme pondérée des distances euclidiennes au carré :

$$\text{cost}(P, C) = \sum_{i=1}^n w_i \min_{c \in C} \|p_i - c\|^2.$$

Cette pondération est essentielle. Un représentant de poids w doit avoir le même effet sur le calcul des centres que w observations identiques de poids unitaire. Ignorer les poids reviendrait à donner la même importance à deux représentants pouvant pourtant résumer des nombres très différents d’observations.

Dans notre implémentation, cette propriété est vérifiée explicitement par les tests unitaires. L’algorithme conserve également le centre précédent lorsqu’un cluster devient vide au cours d’une itération. Ce choix évite d’introduire une réinitialisation supplémentaire susceptible de perturber la convergence.

L’itération s’arrête lorsque l’amélioration relative du coût devient inférieure à un seuil donné ou lorsque le nombre maximal d’itérations est atteint.

2.2 Initialisation k-means++

L’algorithme de Lloyd est sensible aux centres utilisés lors de son initialisation. Une sélection purement aléatoire peut conduire à des centres initiaux concentrés dans une même région des données et, par conséquent, à un minimum local de mauvaise qualité.

L’initialisation k-means++ vise à répartir davantage les centres initiaux. Après le choix d’un premier centre, chaque nouveau centre est sélectionné avec une probabilité proportionnelle au carré de sa distance au centre déjà choisi le plus proche.

Pour un point p , on définit :

$$D(p)^2 = \min_{c \in C} \|p - c\|^2.$$

Dans le cas pondéré utilisé par StreamKM++, la probabilité de sélectionner un point p_i comme nouveau centre est proportionnelle à :

$$w_i D(p_i)^2.$$

Les représentants ayant une masse importante et étant éloignés des centres déjà sélectionnés ont ainsi davantage de chances d’être choisis.

Choix du premier centre. La formulation classique de k-means++ sélectionne le premier centre uniformément parmi les observations. Dans notre implémentation, lorsque l’algorithme est appliqué à un ensemble pondéré, le premier centre est tiré proportionnellement au poids du point.

Les deux stratégies sont équivalentes lorsque tous les poids valent 1. Dans le cas d’un coresets, les poids peuvent en revanche être très différents. Le choix retenu permet donc de tenir compte dès la première sélection de la quantité de données représentée par chaque point du résumé. Il s’agit d’un choix d’implémentation par rapport à la formulation utilisée dans la thèse de référence.

Pour limiter la sensibilité à l’initialisation, l’ensemble du processus k-means++ suivi de Lloyd est exécuté plusieurs fois avec des initialisations différentes. Le modèle présentant le coût final le plus faible est ensuite conservé. Dans nos expériences, cinq redémarrages sont utilisés sauf indication contraire.

2.3 Construction du coresets

Le principal objectif de StreamKM++ est d’éviter de conserver l’ensemble du flux tout en maintenant suffisamment d’information pour produire un clustering de qualité. Cette réduction est réalisée au moyen d’un coresets.

Un coreset peut être vu comme un ensemble pondéré plus petit qui résume les principales caractéristiques géométriques des données originales. Chaque représentant est associé à un poids correspondant à la masse des observations qu’il représente.

La méthode utilisée dans StreamKM++ construit ce résumé à l’aide d’un *coreset tree*. L’idée générale consiste à organiser progressivement les données dans un arbre binaire. Chaque feuille représente un sous-ensemble des observations et possède un point représentatif.

Au départ, une seule feuille contient l’ensemble des points considérés. À chaque étape, une feuille est choisie pour être raffinée. La probabilité de sélectionner une branche dépend du coût associé aux points qu’elle contient : une région mal représentée par son représentant possède un coût plus élevé et a donc davantage de chances d’être divisée.

Une fois la feuille sélectionnée, un nouveau représentant est choisi à l’aide d’un échantillonnage basé sur la distance au représentant courant. Les observations sont ensuite réparties entre les deux représentants selon leur proximité. Ce processus est répété jusqu’à atteindre la taille de coresets souhaitée ou jusqu’à ce qu’aucune subdivision supplémentaire ne soit possible.

Coût et masse. Deux notions doivent être distinguées dans la construction de l’arbre.

La première est le **coût** d’un nœud, défini à partir des distances entre ses observations et son représentant. Cette valeur est utilisée pour décider quelles régions de l’arbre doivent être raffinées.

La seconde est la **masse** du nœud, correspondant à la somme des poids des observations qu’il représente. Cette masse devient le poids du représentant lorsqu’il est finalement placé dans le coresets.

Dans notre implémentation, ces deux grandeurs sont représentées séparément afin d’éviter toute ambiguïté. Le coût intervient dans la sélection des branches à subdiviser, tandis que la masse est utilisée pour pondérer les représentants du coresets.

Conservation de la masse. Une propriété essentielle de la construction est la conservation de la masse totale. Si le jeu d’entrée représente une masse totale

$$W = \sum_i w_i,$$

alors le coresets obtenu doit vérifier :

$$\sum_{q \in S} w_q = W.$$

Autrement dit, la réduction du nombre de représentants ne doit pas entraîner de perte d’information sur la quantité totale de données représentée. Cette propriété constitue l’un des invariants vérifiés dans les tests de notre implémentation.

Cas dégénérés. La taille m constitue une taille cible maximale. Dans le cas général, la construction cherche à produire m représentants. Certaines configurations peuvent toutefois rendre une nouvelle subdivision impossible. C’est notamment le cas lorsque plusieurs points sont identiques et que le coût de la feuille devient nul.

Dans cette situation, le coresets peut contenir moins de m représentants sans que cela constitue une erreur. Les propriétés importantes sont alors que sa taille reste bornée par m et que la masse totale soit correctement conservée.

2.4 Merge-and-reduce

Le coreset tree permet de résumer un ensemble fini de points. Pour traiter un flux dont la taille totale n'est pas connue à l'avance, StreamKM++ lui associe une stratégie *merge-and-reduce*.

Le principe consiste à maintenir une série de buckets :

$$B_0, B_1, B_2, \dots$$

Le premier bucket, B_0 , reçoit directement les observations du flux. Lorsqu'il atteint la taille m , son contenu est propagé vers les niveaux supérieurs.

Si le bucket suivant est vide, le résumé obtenu y est stocké. S'il contient déjà un coreset, les deux ensembles sont réunis puis réduits à nouveau à l'aide du coreset tree. Cette propagation se poursuit tant que le niveau suivant est occupé.

Le mécanisme est analogue à une retenue dans une addition binaire : plusieurs petits résumés sont progressivement fusionnés pour représenter des portions de plus en plus importantes du flux.

Invariants de la structure. Le bucket B_0 contient au plus m points en attente de réduction. Les buckets de niveau supérieur contiennent des résumés pondérés dont la taille reste bornée par m .

Dans la formulation générale de l'algorithme, chaque réduction cherche à produire un coreset de taille m . Notre implémentation autorise toutefois une taille inférieure dans les cas dégénérés décrits précédemment. L'invariant utilisé ne porte donc pas sur l'obtention systématique d'exactly m représentants, mais sur deux propriétés :

- la taille des résumés reste bornée par m ;
- la masse totale représentée est conservée au cours des fusions.

Cette distinction permet notamment de gérer correctement des ensembles comportant de nombreux points identiques.

Coreset courant. À tout instant, l'union des buckets non vides constitue un résumé pondéré de toutes les observations déjà consommées. Le nombre de niveaux augmente progressivement avec la taille du flux, ce qui permet de maintenir une consommation mémoire contrôlée sans avoir besoin de connaître à l'avance le nombre total d'observations.

Lorsqu'un clustering est demandé, les représentants présents dans les différents buckets sont rassemblés afin de former le coreset courant. K-means++ et Lloyd peuvent alors être appliqués sur ce résumé.

Fusion de deux résumés indépendants. La propriété de composition des coresets permet également de fusionner des résumés construits indépendamment sur des sous-ensembles distincts des données. Les représentants des deux résumés peuvent être réunis puis réduits à nouveau pour obtenir un coreset de leur union.

Cette propriété est particulièrement importante pour le portage vers Spark. Chaque partition peut construire indépendamment son propre résumé à l'aide d'un **BucketManager**. Les résumés produits par les partitions sont ensuite fusionnés progressivement pour obtenir un coreset global.

L'algorithme de StreamKM++ reste ainsi contenu dans les couches `streamkm.core` et `streamkm.coreset`. La couche Spark n'a pas besoin de modifier la logique du coreset : elle organise uniquement l'exécution de ces opérations sur plusieurs partitions.

Chapitre 3

Apache Spark

La thèse de référence présente d’abord le modèle d’exécution d’Apache Flink avant de décrire l’implémentation de StreamKM++. Nous suivons la même logique avec Apache Spark. Ce chapitre présente uniquement les mécanismes du framework qui sont mobilisés par notre solution : les RDD et DataFrames, le traitement par micro-batches de Structured Streaming, la diffusion de données avec `broadcast`, la réduction arborescente et le partitionnement.

L’objectif n’est pas de proposer une présentation générale de Spark, mais de mettre en évidence les propriétés qui influencent directement le portage de StreamKM++. Leur utilisation concrète dans le projet est décrite au chapitre 4.

3.1 RDD et DataFrames

Apache Spark fournit plusieurs abstractions pour manipuler des données distribuées. Deux d’entre elles sont utilisées dans notre implémentation : les *Resilient Distributed Datasets* (RDD) et les DataFrames.

Un RDD est une collection distribuée et partitionnée d’objets. Les opérations appliquées à un RDD, telles que `map`, `filter` ou `mapPartitions`, sont exécutées indépendamment sur ses différentes partitions lorsque cela est possible. Cette abstraction convient au traitement des objets `Point` utilisés par StreamKM++, car l’algorithme repose principalement sur des opérations impératives locales : calcul de distances, construction de coresets et mise à jour des structures merge-and-reduce.

Les DataFrames représentent quant à eux des données distribuées organisées selon un schéma. Spark peut optimiser les opérations qui leur sont appliquées à travers son moteur d’exécution SQL, notamment lorsque les traitements reposent sur des expressions, des projections ou des agrégations structurées.

Dans notre architecture, les DataFrames sont principalement utilisés aux frontières du traitement, notamment avec Structured Streaming, tandis que la logique algorithmique de StreamKM++ est exécutée sur des RDD de `Point`. Cette séparation permet de conserver les composants algorithmiques indépendants de Spark tout en exploitant les mécanismes de distribution du framework.

3.2 Traitement par micro-batches avec Structured Streaming

Apache Spark Structured Streaming permet de traiter des flux de données à l’aide de la même API que les traitements distribués classiques. Dans notre implémentation, il est utilisé en mode *micro-batch*.

Le flux d’entrée est alors découpé en lots successifs. Chaque micro-batch est considéré comme un ensemble de données borné et peut être traité au moyen des opérations Spark habituelles.

La méthode `foreachBatch` permet notamment d'appliquer une fonction spécifique à chaque lot produit par le moteur.

Ce modèle diffère de celui de Flink utilisé dans la thèse, où le traitement est organisé autour d'opérateurs de streaming continus et d'un état maintenu au niveau des subtasks. Dans l'implémentation étudiée par Bitsakis, certaines frontières du traitement doivent notamment être détectées au moyen de mécanismes propres à Flink, tels que les watermarks.

Avec Structured Streaming, la frontière entre deux micro-batches est explicitement fournie par le moteur d'exécution. Cette caractéristique simplifie certaines étapes du portage, puisqu'un lot peut être traité comme un ensemble borné avant que le résultat intermédiaire soit intégré à l'état global de l'application.

Cette approche introduit toutefois une différence importante avec un traitement événement par événement : les observations ne sont pas nécessairement traitées immédiatement lors de leur arrivée, mais lors de l'exécution du micro-batch auquel elles appartiennent. Ce compromis est discuté plus en détail au chapitre 6.

3.3 Diffusion des centres avec broadcast

Certaines opérations nécessitent qu'une même donnée soit accessible à l'ensemble des tâches Spark. C'est notamment le cas lors de l'évaluation de la qualité d'un clustering : les centres calculés doivent être utilisés par toutes les partitions contenant les observations originales.

Spark fournit pour cela le mécanisme de *broadcast variables*. Une valeur diffusée est transmise aux exécuteurs puis réutilisée localement par les tâches qui en ont besoin, ce qui évite de sérialiser et de transférer la même information à chaque opération.

Dans notre projet, ce mécanisme est utilisé par `CostEvaluator.cost`. Les centres du clustering sont diffusés, puis chaque partition calcule localement sa contribution à la somme des erreurs quadratiques. Les résultats partiels sont ensuite agrégés afin d'obtenir le coût global.

Cette organisation correspond au rôle joué dans la thèse par la diffusion des centres vers les opérateurs chargés de calculer le coût sur les données distribuées. L'utilisation concrète de cette opération est détaillée dans la section 4.2.

3.4 Réduction arborescente

La construction distribuée de `StreamKM++` produit plusieurs résumés intermédiaires, généralement un par partition. Ces résultats doivent ensuite être combinés pour former un coreset global.

Une première approche consisterait à transférer tous les résumés vers le driver puis à les fusionner séquentiellement. Cette solution concentre le travail de réduction sur un seul processus et devient moins adaptée lorsque le nombre de partitions augmente.

Spark fournit des opérations de réduction arborescente telles que `treeReduce` et `treeAggregate`. Au lieu d'effectuer toutes les combinaisons directement sur le driver, les résultats intermédiaires sont fusionnés progressivement à plusieurs niveaux. Une partie du travail peut ainsi être réalisée en parallèle avant que le résultat final ne soit renvoyé au driver.

Ce mécanisme est particulièrement adapté au merge-and-reduce de `StreamKM++`, car la fusion de deux résumés produit elle-même un résumé qui peut être fusionné à un niveau supérieur.

Dans notre implémentation, `treeReduce` est utilisé pour combiner les structures `BucketManager` construites indépendamment sur les différentes partitions. Ce choix permet de conserver une graine propre à chaque partition lors de la construction des coresets locaux et d'éviter l'introduction d'un accumulateur initial partagé entre les partitions.

L'intérêt de cette réduction arborescente est principalement architectural : elle évite de centraliser immédiatement toutes les opérations de fusion sur le driver. Son impact réel sur le temps

d'exécution doit toutefois être mesuré expérimentalement et sera étudié dans la section consacrée à la scalabilité.

3.5 Partitions et ressources de calcul

Le partitionnement constitue un élément central du modèle d'exécution Spark. Un RDD est divisé en plusieurs partitions, chacune pouvant être traitée indépendamment par une tâche.

Dans notre cas, chaque partition construit son propre `BucketManager` et produit donc un résumé local des observations qui lui sont attribuées. Le nombre de partitions influence ainsi directement la structure de la construction distribuée : modifier ce nombre modifie le nombre de coreset intermédiaires ainsi que la manière dont ils sont ensuite fusionnés.

Il est important de distinguer ce paramètre du nombre de ressources de calcul disponibles. Le nombre de partitions définit le nombre potentiel de tâches et la décomposition logique des données, tandis que le nombre de cœurs disponibles détermine combien de ces tâches peuvent effectivement être exécutées simultanément.

Les deux paramètres ne doivent donc pas être confondus. Une augmentation du nombre de partitions n'implique pas automatiquement une amélioration des performances : un partitionnement plus fin peut augmenter les possibilités de parallélisme, mais également introduire davantage de tâches, de sérialisation et d'opérations de fusion.

De la même manière, le partitionnement peut influencer le coreset final, puisque chaque partition réalise sa propre construction locale avec une graine déterminée. L'effet sur la qualité du clustering n'est donc pas supposé a priori ; il doit être observé expérimentalement.

Cette séparation entre partitionnement des données et ressources de calcul constitue un point intéressant du portage vers Spark. Elle permet d'étudier séparément l'effet du découpage des données et celui du parallélisme matériel, ce qui sera exploité dans les expériences de scalabilité du chapitre 5.

Chapitre 4

Implémentation

Ce chapitre décrit la couche `streamkm.spark`, opérateur par opérateur, en suivant le même plan que le chapitre 4 de la thèse : d’abord le dataflow borné (§4.1, miroir du §4.1 de la thèse), puis la méthodologie d’évaluation du coût (§4.2, miroir du §4.2), puis la variante à requêtes périodiques (§4.3, miroir du §4.3, *décrite* plutôt qu’implémentée — cf. justification en fin de section), et enfin un bilan de ce qui se traduit ou non depuis Flink (§4.4). Rappel de la règle d’or (chapitre 1) : aucun des fichiers décrits ici ne contient de logique algorithmique propre — tout le calcul vient de `streamkm.core` et `streamkm.coreset` (chapitre 2), déjà testés indépendamment de Spark.

4.1 StreamKM++ distribué, flux borné

4.1.1 Coresets partiels par partition

`StreamKMPlusPlus.partialCoresets` porte l’opérateur `FlatMapToPartialCoresets` de la thèse (§4.1) : chaque subtask Flink « consomme les points d’entrée selon l’algorithme 2.5 » — une partition Spark joue exactement ce rôle.

```
def partialCoresets(rdd: RDD[Point], p: StreamKMParams): RDD[Array[Point]] =  
  rdd.mapPartitionsWithIndex { (idx, it) =>  
    val bm = new BucketManager(p.m, p.seed + idx)  
    bm.insertAll(it)  
    Iterator.single(bm.extractCoreset())  
  }
```

La graine de chaque `BucketManager` est dérivée de l’indice de partition (`p.seed + idx`) plutôt que partagée entre partitions — condition nécessaire pour que l’expérience E3bis (chapitre 5) reste interprétable : avec une graine partagée, changer le nombre de partitions changerait aussi la suite aléatoire consommée par chaque partition, confondant l’effet mesuré (le partitionnement) avec un simple changement de graine.

4.1.2 Fusion arborescente du coreset final

`StreamKMPlusPlus.mergeCoresets` remplace l’opérateur `FlatMapToFinalCoreset` de la thèse — l’opérateur **non parallèle** par construction (§4.1 : fusion séquentielle, un coreset partiel à la fois) que la thèse elle-même identifie comme le goulot qui plafonne le speedup au-delà de 8 cœurs (§5.3.1). C’est notre apport principal par rapport à la thèse, rendu possible par le découplage partitions/cœurs de Spark (§3.4) :

```
def mergeCoresets(rdd: RDD[Point], p: StreamKMParams, depth: Int = 2): Array[Point] = {
```

```

val perPartition: RDD[BucketManager] = rdd.mapPartitionsWithIndex { (idx, it) =>
  val bm = new BucketManager(p.m, p.seed + idx)
  bm.insertAll(it)
  Iterator.single(bm)
}
perPartition.treeReduce((a, b) => a.merge(b), depth).extractCoreset()
}

```

Le `combOp` de ce `treeReduce` est `BucketManager.merge` (chapitre 2, §??), réutilisé *tel quel* — aucune extension de `streamkm.coreset` n’a été nécessaire pour distribuer merge-and-reduce sur un cluster, ce qui confirme a posteriori la séparation stricte des couches annoncée en introduction.

4.1.3 Boucle Structured Streaming complète

`StreamKMPlusPlus.run` assemble l’ensemble du dataflow — de l’équivalent de `HdfsSource` jusqu’à l’équivalent de `FlatMapToKmeansPP` — autour d’un état global unique (`BucketManager`) maintenu côté *driver* entre les micro-batches :

```

def run(source: DataFrame, p: StreamKMParams,
        onModel: KMeansModel => Unit,
        featuresCol: String = "features"): StreamingQuery = {

  val global = new BucketManager(p.m, p.seed)

  source.writeStream.foreachBatch { (batch, batchId) =>
    val points = batch.rdd.map(row => Point(row.getAs[Seq[Double]](featuresCol).toArray))
    val partial = mergeCoresets(points, p.copy(seed = p.seed + batchId))
    partial.foreach(global.insert)
    val model = KMeans.fit(global.extractCoreset(), p.k, p.nRestarts, seed = p.seed)
    onModel(model)
  }.start()
}

```

Chaque micro-batch produit son propre coreset distribué (`mergeCoresets`), qui est ensuite intégré dans l’état global par `insert` — exactement la même opération de fusion que `FlatMapToFinalCoreset`, appliquée cette fois entre micro-batches plutôt qu’entre partitions. Le contrat d’entrée (une colonne `featuresCol` de type `array<double>`, poids implicite 1.0) reflète directement `Point.scala` (chapitre 2) : le poids n’existe que dans les coresets produits en interne, jamais dans les données brutes du flux.

4.1.4 Table de correspondance

4.2 Méthodologie d’évaluation du coût

La thèse (§4.2) recalcule le coût d’un clustering en parallèle, à partir d’un fichier de centres déjà connu, dans un job Flink séparé — nécessaire pour mesurer la qualité sur de gros volumes sans que ce calcul devienne lui-même le goulot d’étranglement des expériences du chapitre 5. Elle utilise pour cela un *connected stream* (deux sources fusionnées : les points et les centres), un opérateur de diffusion et un opérateur de somme partielle. Cette machinerie tient, côté Spark, en trois lignes :

```

def cost(points: RDD[Point], centers: Array[Array[Double]], depth: Int = 2): Double = {
  val bcCenters = points.sparkContext.broadcast(centers)

```

Opérateur (thèse)	Flink	Rôle	Équivalent Spark
HdfsSource		lecture parallèle des lignes	source fichier partitionnée
HandleWatermark		détecter la fin de fichier	aucun — <code>foreachBatch</code> rend la frontière explicite
FlatMapToPoint		parsing <code>String</code> $\rightarrow$ <code>Point</code>	<code>mapPartitions</code> + <code>filter</code>
FlatMapToPartialCoresets		merge-and-reduce par subtask	<code>partialCoresets</code>
FlatMapToFinalCoreset		fusion séquentielle (par. 1)	<code>mergeCoresets</code> (<code>treeReduce</code>)
FlatMapToKmeansPP		$5 \times k$ -means++ (par. 1)	<code>KMeans.fit</code> sur le driver
HdfsSink		écriture des centres	sink (fichier ou <code>onModel</code>)

TABLE 4.1 – Correspondance des opérateurs, dataflow borné (§4.1 de la thèse).

```

points
  .mapPartitions(it => Iterator.single(KMeans.cost(it.toArray, bcCenters.value)))
  .treeReduce(_ + _, depth)
}

```

`sc.broadcast(centers)` remplace `FlatMapToCentroid` + `.broadcast()`; `mapPartitions(... KMeans.cost ...)` remplace `CoFlatMapToPartialCost`; `treeReduce(_+_)` remplace `CoFlatMapToFinalCost`. Contrairement à la fusion de coresets (§4.1), cette agrégation est une simple somme — associative et exacte à la précision flottante près — ce qui explique pourquoi elle ne nécessite aucune des précautions de reproductibilité par partition discutées plus haut : le résultat ne dépend pas du nombre de partitions choisi. C’est cet instrument qui sert de mesure à **toutes** les expériences du chapitre 5 — jamais le coût mesuré sur le coreset lui-même, qui n’est pas comparable au coût sur les données complètes.

4.3 Variante à requêtes périodiques (décrite, non implémentée)

Le §4.3 de la thèse propose une variante où le clustering est ré-évalué périodiquement plutôt qu’une seule fois à la fin du flux, et où plusieurs valeurs de k sont calculées simultanément. Faute de temps avant la deadline, cette variante **n’a pas été implémentée** — conformément à notre méthodologie de travail, qui préférerait explicitement documenter cette limite plutôt que de livrer une implémentation bâclée. La thèse elle-même ne l’évalue pas expérimentalement (elle le signale comme travail futur, ch. 6), ce qui rend ce choix d’autant plus défendable.

Le portage resterait néanmoins direct avec ce que nous avons déjà construit : le *trigger* de Structured Streaming remplacerait directement l’opérateur `SourceRequests_R` de génération de requêtes périodiques (`Trigger.ProcessingTime("30 seconds")` au lieu d’un flux de requêtes explicite); `BucketManager.extractCoreset` — qui ne vide jamais l’état, une propriété déjà exploitée par `run` — permettrait de dupliquer le coreset courant pour plusieurs valeurs de k simultanément (`Seq(25, 50, 75).map(k => KMeans.fit(coreset, k))`), remplaçant `FlatMapToFinalCoreset_R`. Seule la persistance du meilleur clustering *par* k (`ValueState` chez Flink) demanderait une structure nouvelle côté Spark (une simple `Map[Int, KMeansModel]` tenue par le `onModel` de `run` suffirait).

Le §4.3.1 de la thèse (*Queryable State*, interrogation en temps réel de l’état interne depuis l’extérieur du job) **n’a en revanche pas d’équivalent Spark** : Spark n’expose aucun état inter-micro-batch interrogeable depuis l’extérieur. C’est un avantage architectural réel de Flink, que nous assumons comme tel — voir la discussion du chapitre 6. Le contournement usuel (un sink `memory` interrogé par `spark.sql("SELECT * FROM centers")`, ou l’écriture des centres dans un store externe) n’offre pas la même garantie de fraîcheur immédiate.

4.4 Ce qui se traduit depuis Flink, ce qui ne se traduit pas

Ce paragraphe de synthèse répond directement à la question posée par l'énoncé — porter en Spark une solution Flink — et n'a pas d'équivalent dans la thèse : c'est notre contribution propre.

- **Disparaît entièrement** : `HandleWatermark`. Le bricolage watermark/marqueur « EOF » de la thèse n'a plus de raison d'être — `foreachBatch` rend la frontière de micro-batch explicite par construction (§3).
- **Se traduit avec un gain net** : la fusion des coreset partiels. L'opérateur non-parallèle de la thèse (§4.1) devient une réduction en arbre (`treeReduce`), ce qui repousse directement le plafond de speedup identifié au §5.3.1 de la thèse — à confirmer expérimentalement (chapitre 5, expérience E7).
- **Se traduit à l'identique, en plus concis** : la méthodologie d'évaluation du coût (§4.2) — trois opérateurs Flink dédiés et un *connected stream* deviennent trois lignes de `broadcast + treeReduce`.
- **Se traduit, avec un degré de liberté nouveau** : le parallélisme. Le couplage partitions/cœurs de Flink se découple côté Spark (§3.5), ce qui permet une expérience (E3bis) que le modèle de Flink ne permet pas.
- **Ne se traduit pas** : Queryable State (§4.3). C'est la seule perte de fonctionnalité nette identifiée dans ce portage — discutée au chapitre 6.

Chapitre 5

Évaluation expérimentale

Ce chapitre présente l'évaluation expérimentale de l'implémentation de StreamKM++ réalisée dans le cadre du projet. L'objectif est d'abord de vérifier la correction des principales briques algorithmiques, puis d'étudier la qualité du clustering obtenu sur un jeu de données réel. La scalabilité de l'implémentation Spark est ensuite évaluée afin d'observer son comportement lorsque le volume de données et le degré de parallélisme augmentent.

L'évaluation suit volontairement cet ordre. Mesurer les performances d'une implémentation distribuée n'aurait en effet que peu de sens si les résultats produits n'étaient pas préalablement validés.

5.1 Protocole expérimental

Les expériences principales sont réalisées sur le jeu de données Coverttype. Ce jeu contient 581 012 observations décrites par 54 attributs numériques utilisés pour le clustering. La dernière colonne, correspondant à la classe associée à chaque observation, n'est pas utilisée, StreamKM++ étant un algorithme de clustering non supervisé.

Les expériences distribuées utilisent Apache Spark avec huit partitions. Chaque partition traite une partie des observations et construit un coreset local. Les résumés produits sont ensuite fusionnés afin d'obtenir un coreset global sur lequel le clustering final est exécuté.

Afin de rendre les différentes configurations comparables, une graine fixe égale à 2026 est utilisée. Chaque configuration présentée dans cette section a été exécutée une fois. Les résultats permettent ainsi de comparer les différentes approches dans des conditions reproductibles, mais ne permettent pas de mesurer précisément la variabilité induite par les étapes aléatoires de StreamKM++ et de l'initialisation k-means++.

La qualité du clustering est évaluée à partir de la somme des erreurs quadratiques, ou *Sum of Squared Errors* (SSE). Pour un ensemble de données X et un ensemble de centres C , cette mesure est définie par :

$$\text{SSE}(X, C) = \sum_{x \in X} \min_{c \in C} \|x - c\|^2.$$

Plus cette valeur est faible, plus les observations sont proches de leur centre associé.

Dans les expériences présentées ici, le coreset est utilisé uniquement pour déterminer les centres. La SSE est ensuite calculée sur l'ensemble des données originales. Cette distinction est importante afin d'évaluer la qualité réelle du clustering obtenu sur le jeu complet et non uniquement sur son résumé.

Trois approches sont comparées :

- StreamKM++ séquentiel ;
- StreamKM++ distribué avec Apache Spark ;
- K-Means de Spark MLlib.

La version séquentielle fournit une référence interne permettant de vérifier que la distribution de la construction du coreset ne modifie pas fortement la qualité du résultat. Spark MLlib constitue quant à lui une référence externe. Contrairement à StreamKM++, MLlib applique K-Means directement aux données distribuées et ne repose pas sur la construction préalable d'un coreset. La comparaison porte donc principalement sur la qualité des centres obtenus, et non sur une équivalence stricte entre les deux approches.

5.2 Validation de l'implémentation

Avant d'exécuter les expériences sur Coverttype, plusieurs tests déterministes ont été réalisés afin de vérifier séparément les principales briques de l'implémentation.

Cette étape est nécessaire car une erreur dans le calcul des distances, dans la prise en compte des poids ou dans les opérations de fusion pourrait conduire à des résultats numériquement plausibles tout en étant incorrects.

Les principaux tests réalisés sont résumés dans le tableau 5.1.

Test	Résultat attendu	Résultat obtenu
Distance euclidienne au carré	25	25
K-Means déterministe	SSE = 4	SSE = 4
K-Means pondéré	centre (2.5, 0), SSE = 75	conforme
Construction du coreset	masse conservée	conforme
BucketManager	masse conservée	conforme
Lecture de Coverttype	54 dimensions	conforme

TABLE 5.1 – Validation des principales briques de l'implémentation.

Le premier test porte sur le calcul de la distance euclidienne au carré. Il utilise deux points pour lesquels le résultat attendu peut être déterminé manuellement. La valeur obtenue, égale à 25, correspond à la valeur attendue.

Le deuxième test vérifie le fonctionnement de l'algorithme de Lloyd sur un petit jeu de données déterministe. Le coût final obtenu est égal à 4, conformément au résultat attendu.

Un test spécifique vérifie également la prise en compte des poids. Pour le jeu de données considéré, le centre attendu est (2.5, 0) et la SSE attendue est égale à 75. Le résultat obtenu est conforme.

Les tests sur la construction du coreset et sur le **BucketManager** vérifient quant à eux la conservation de la masse. La somme des poids des points contenus dans le résumé doit rester égale au nombre d'observations représentées, même après plusieurs opérations de réduction et de fusion.

Enfin, le chargement de Coverttype a été vérifié afin de s'assurer que chaque observation est correctement transformée en un vecteur de 54 dimensions après suppression de la variable de classe.

Ces tests ne constituent pas une mesure de la qualité du clustering à grande échelle. Ils permettent cependant de vérifier séparément les propriétés essentielles de l'implémentation avant son évaluation sur le jeu de données complet.

5.3 Qualité du clustering sur Coverttype

Les expériences de qualité sont réalisées sur les 581012 observations de Coverttype. Deux paramètres sont étudiés : le nombre de clusters k et la taille maximale m du coreset.

L'objectif est d'observer d'une part l'évolution de la SSE lorsque le nombre de centres augmente, et d'autre part l'effet de la taille du coreset sur la qualité du clustering.

5.3.1 Influence du nombre de clusters

La première expérience fait varier le nombre de clusters selon :

$$k \in \{10, 20, 30, 40, 50\}.$$

Pour chaque valeur de k , la taille du coreset est définie par :

$$m = 200k.$$

Le tableau 5.2 présente les résultats obtenus avec les trois méthodes.

k	StreamKM++ séquentiel	StreamKM++ Spark	Spark MLlib
10	3.400×10^{11}	3.393×10^{11}	3.388×10^{11}
20	2.027×10^{11}	2.040×10^{11}	2.023×10^{11}
30	1.546×10^{11}	1.545×10^{11}	1.533×10^{11}
40	1.298×10^{11}	1.293×10^{11}	1.281×10^{11}
50	1.144×10^{11}	1.139×10^{11}	1.125×10^{11}

TABLE 5.2 – Influence du nombre de clusters sur la SSE pour les 581 012 observations de Coverttype, avec $m = 200k$.

Pour les trois méthodes, la SSE diminue nettement lorsque le nombre de clusters augmente. Elle passe d'environ 3.4×10^{11} pour $k = 10$ à environ 1.1×10^{11} pour $k = 50$.

Ce comportement est attendu dans le cadre de K-Means. L'augmentation du nombre de centres permet une représentation plus fine des données et réduit ainsi la distance moyenne entre les observations et leur centre le plus proche.

Les résultats obtenus par les versions séquentielle et distribuée de StreamKM++ sont particulièrement proches. Pour toutes les valeurs de k testées, l'écart relatif entre leurs coûts reste inférieur à 0.7 %.

Pour $k = 10$, la version Spark obtient même une SSE légèrement inférieure à celle de la version séquentielle. Pour $k = 20$, son coût est légèrement supérieur, puis les deux approches deviennent à nouveau très proches pour les valeurs plus élevées de k .

Ces écarts peuvent notamment provenir de l'ordre de construction et de fusion des coresets ainsi que des étapes aléatoires intervenant dans l'algorithme. Ils restent néanmoins faibles au regard du coût total observé.

Ces résultats indiquent que la distribution de la construction du coreset entre les partitions Spark ne conduit pas, dans les configurations testées, à une dégradation notable de la qualité du clustering par rapport à l'implémentation séquentielle.

5.3.2 Comparaison avec Spark MLlib

Les résultats du tableau 5.2 montrent également que les coûts obtenus avec Spark MLlib sont proches de ceux de StreamKM++.

Pour toutes les valeurs de k testées, MLlib produit une SSE légèrement inférieure à celle de StreamKM++ Spark. L'écart reste toutefois limité. Pour $k = 10$, il est d'environ 0.16 %, tandis qu'il atteint environ 1.24 % pour $k = 50$.

Cette différence doit être interprétée en tenant compte du fonctionnement des deux approches. Spark MLlib applique K-Means directement aux observations distribuées, tandis que StreamKM++ construit d'abord un résumé pondéré de taille limitée avant de déterminer les centres.

La réduction des données introduit donc nécessairement une approximation supplémentaire. Malgré cela, les valeurs obtenues restent proches de celles de MLlib sur les configurations testées.

Cette expérience suggère ainsi que le coreset conserve suffisamment d’information pour produire des centres dont la qualité reste comparable à celle obtenue par une exécution directe de K-Means sur les données distribuées.

5.3.3 Influence de la taille du coreset

La deuxième série d’expériences étudie l’influence de la taille du coreset. Le nombre de clusters est fixé à :

$$k = 25,$$

et les tailles suivantes sont testées :

$$m \in \{1000, 5000, 10000, 20000\}.$$

Les résultats sont présentés dans le tableau 5.3.

m	StreamKM++ séquentiel	StreamKM++ Spark
1 000	1.807×10^{11}	1.820×10^{11}
5 000	1.748×10^{11}	1.759×10^{11}
10 000	1.746×10^{11}	1.745×10^{11}
20 000	1.742×10^{11}	1.740×10^{11}

TABLE 5.3 – Influence de la taille du coreset pour $k = 25$ sur les 581 012 observations de Coverture.

L’augmentation de la taille du coreset s’accompagne globalement d’une diminution de la SSE.

Pour la version séquentielle, le coût passe de 1.807×10^{11} avec $m = 1000$ à 1.742×10^{11} avec $m = 20000$. La version Spark suit une évolution similaire, avec une diminution de 1.820×10^{11} à 1.740×10^{11} .

La diminution la plus importante est observée entre $m = 1000$ et $m = 5000$. Pour des tailles plus élevées, l’amélioration devient progressivement plus faible.

Ce résultat est cohérent avec le rôle du coreset. Un résumé de plus grande taille conserve davantage d’information sur la distribution des observations et peut ainsi fournir une approximation plus précise du problème initial. Cependant, au-delà d’une certaine taille, le gain supplémentaire semble devenir limité.

Il apparaît donc un compromis entre la quantité d’information conservée dans le coreset et la taille du résumé manipulé lors des étapes suivantes.

Les versions séquentielle et Spark restent par ailleurs très proches pour toutes les valeurs de m . L’écart maximal observé est inférieur à 0.8 %. Pour $m = 10000$ et $m = 20000$, les coûts obtenus par les deux implémentations sont pratiquement identiques.

Ces résultats renforcent l’observation effectuée lors de l’étude de k : la construction distribuée du coreset fournit une approximation dont la qualité reste très proche de celle obtenue avec l’implémentation séquentielle.

5.3.4 Limites de l’évaluation de qualité

Les expériences précédentes permettent de comparer les différentes approches sur l’intégralité du jeu Coverture, mais plusieurs limites doivent être prises en compte.

La première concerne le nombre d’exécutions. Chaque configuration a été testée une seule fois avec une graine fixée à 2026. Ce choix garantit que les comparaisons sont reproductibles, mais ne permet pas de mesurer la variabilité liée aux étapes aléatoires de l’algorithme.

Une évaluation plus complète pourrait répéter chaque configuration avec plusieurs graines et présenter la moyenne, l'écart-type ainsi que les valeurs minimales et maximales obtenues.

La seconde limite concerne la métrique utilisée. La SSE permet de mesurer directement la compacité des clusters, mais elle ne fournit pas à elle seule une mesure absolue de la pertinence du partitionnement. Dans le cadre de ce projet, elle est principalement utilisée pour comparer les différentes implémentations dans des conditions identiques.

5.4 Scalabilité de l'implémentation Spark

Après avoir vérifié la qualité des résultats produits, l'évaluation porte sur les performances de l'implémentation distribuée.

L'objectif est d'étudier l'évolution du temps d'exécution lorsque le volume de données augmente et de déterminer dans quelle mesure la construction distribuée des coresets peut exploiter le parallélisme offert par Spark.

Deux axes expérimentaux sont considérés.

Le premier consiste à faire varier le nombre d'observations traitées. Les volumes envisagés sont :

$$n \in \{20\,000, 50\,000, 100\,000, 200\,000, 581\,012\}.$$

Le second consiste à faire varier le nombre de partitions Spark :

$$p \in \{1, 2, 4, 8, 16\}.$$

Pour chaque configuration, le temps total d'exécution du pipeline doit être mesuré dans des conditions identiques.

Ces expériences doivent permettre d'observer le compromis entre le parallélisme obtenu grâce au découpage des données et le surcoût propre à Spark. Celui-ci inclut notamment la création et l'ordonnancement des tâches, les opérations de sérialisation, la gestion des partitions ainsi que la fusion des coresets intermédiaires.

L'objectif n'est donc pas uniquement de rechercher une diminution absolue du temps d'exécution. Il s'agit également d'identifier à partir de quel volume le traitement distribué devient avantageux par rapport au coût d'orchestration introduit par Spark.

5.5 Optimisation du noyau local

En complément des expériences principales, une optimisation de la représentation mémoire des points a été étudiée sur les composants locaux de l'algorithme.

Cette expérimentation ne porte pas directement sur Spark. Elle cherche à déterminer si une représentation plus compacte des coordonnées peut accélérer les opérations fréquemment exécutées lors du clustering et de la construction des coresets.

5.5.1 Hypothèse

L'implémentation initiale utilise une représentation de type *Array of Structures* (AoS), dans laquelle un ensemble de points est représenté par un `Array[Point]` et chaque objet `Point` contient son propre `Array[Double]`.

Une seconde représentation, de type *Structure of Arrays* (SoA), a été développée dans `streamkm.core.PointsSoA`. Les coordonnées sont stockées de manière contiguë dans un `DoubleBuffer` reposant sur un `ByteBuffer.allocateDirect`.

L'hypothèse était qu'une meilleure localité mémoire pouvait réduire le coût des accès aux coordonnées, notamment dans les opérations appelées de manière intensive telles que le calcul des distances, la recherche du centre le plus proche et la construction du coreset.

5.5.2 Protocole JMH

Les deux représentations ont été comparées à l'aide de JMH 1.37.

Les benchmarks ont été exécutés avec cinq itérations de warmup, trente itérations de mesure et un fork. Le même JDK a été utilisé pour les deux versions afin de conserver des conditions de comparaison identiques.

Six opérations ont été évaluées :

- `sqdist` ;
- `nearest` ;
- `BucketManager.insertAll` ;
- `CoresetTree.build` ;
- `KMeans.lloyd` ;
- le pipeline local complet.

Ces microbenchmarks mesurent uniquement les performances du noyau local. Ils ne permettent pas de conclure directement sur les performances d'un job Spark complet.

5.5.3 Résultats

Contrairement à l'hypothèse initiale, la représentation SoA utilisant un buffer hors tas s'est révélée plus lente que la représentation AoS sur l'ensemble des benchmarks.

Benchmark	Nature	Écart SoA vs AoS
<code>sqdist</code>	calcul de distance	+46 à +77 %
<code>nearest</code>	recherche de centre	+45 à +51 %
<code>insertAll</code>	manipulation du buffer	+32 à +50 %
<code>CoresetTree.build</code>	construction du coreset	+36 à +59 %
<code>lloyd</code>	clustering	+20 à +30 %
<code>fullPipeline</code>	pipeline complet	+15 à +21 %

TABLE 5.4 – Écart relatif du temps moyen entre la représentation SoA hors tas et la représentation AoS.

L'écart est le plus important sur les opérations élémentaires directement dépendantes de l'accès aux coordonnées, notamment `sqdist` et `nearest`. Il devient plus faible lorsque le benchmark inclut davantage d'opérations indépendantes de la représentation mémoire, comme dans `lloyd` ou dans le pipeline complet.

Dans l'implémentation testée, le bénéfice attendu de la disposition contiguë des coordonnées ne compense donc pas le coût associé aux accès via `DoubleBuffer`.

Ce résultat ne remet pas nécessairement en cause le principe d'une représentation SoA. Il montre uniquement que la variante fondée sur `ByteBuffer.allocateDirect` et `DoubleBuffer` n'apporte pas d'amélioration dans le contexte étudié.

Une variante reposant sur un tableau `Array[Double]` plat conservé sur le tas pourrait permettre de distinguer l'effet de la disposition SoA de celui du stockage hors tas. Cette expérimentation n'a cependant pas été poursuivie dans le temps imparti au projet.

La représentation initiale a donc été conservée pour les expériences principales.

5.6 Synthèse

Les expériences menées sur Coverttype montrent tout d'abord que les versions séquentielle et distribuée de StreamKM++ produisent des coûts très proches. Pour les différentes valeurs de k étudiées, l'écart entre les deux implémentations reste inférieur à 0.7 %. Lors de la variation de la taille du coreset, l'écart maximal reste inférieur à 0.8 %.

La comparaison avec Spark MLlib montre également que le coût obtenu par StreamKM++ reste proche de celui d'un K-Means exécuté directement sur les données distribuées. La réduction des observations sous la forme d'un coreset n'entraîne donc pas, dans les configurations testées, de dégradation importante de la qualité du clustering.

Les expériences sur la taille du coreset montrent enfin qu'une augmentation de m améliore globalement la qualité du résultat, mais que le gain devient progressivement plus faible lorsque le résumé augmente.

Ces résultats valident ainsi la qualité de l'implémentation avant l'étude de sa scalabilité. Les expériences de performance doivent maintenant déterminer si la construction distribuée des coresets permet également de bénéficier efficacement du parallélisme lorsque le volume de données augmente.

Chapitre 6

Discussion

Ce chapitre met en perspective les choix réalisés dans le portage de StreamKM++ vers Apache Spark. L’objectif n’est pas de reprendre les résultats du chapitre 5, mais d’en dégager les principaux apports et les limites de l’implémentation obtenue.

6.1 Apports de l’implémentation

Une construction du coreset adaptée au traitement de flux. Le principal intérêt de StreamKM++ réside dans sa capacité à construire progressivement un résumé des observations sans conserver l’ensemble du flux en mémoire. Lors de la construction du coreset, les points sont consommés progressivement par le mécanisme merge-and-reduce présenté au chapitre 2.

La quantité de données conservée dépend ainsi principalement de la taille m du coreset et du nombre de niveaux nécessaires dans la structure de buckets, plutôt que directement du nombre total d’observations déjà consommées. Cette propriété permet d’envisager le traitement de flux dont la taille finale n’est pas connue à l’avance.

Il convient toutefois de distinguer la construction du coreset du protocole expérimental utilisé dans ce projet. Pour mesurer la SSE des centres obtenus sur les données originales, les expériences peuvent nécessiter un parcours supplémentaire du jeu de données. Cette seconde lecture appartient à l’évaluation du modèle et non à l’algorithme StreamKM++ lui-même.

Une séparation claire entre logique algorithmique et distribution. L’architecture du projet sépare les composants de clustering des mécanismes propres à Spark. Les packages `streamkm.core` et `streamkm.coreset` contiennent respectivement les opérations liées à k-means et aux coresets, tandis que `streamkm.spark` organise leur exécution distribuée.

Cette séparation a permis de valider les principales propriétés algorithmiques indépendamment de Spark avant leur intégration dans le pipeline distribué. Les mêmes classes `BucketManager`, `CoresetTree` et `KMeans` sont ainsi réutilisées dans les versions séquentielle et distribuée.

Les résultats de validation présentés au chapitre 5 montrent notamment que les versions séquentielle et Spark produisent des coûts très proches sur les configurations étudiées, ce qui constitue un indicateur important de la cohérence du portage.

Une construction naturellement distribuable par partition. Le partitionnement des données proposé par Spark s’intègre bien à la structure de StreamKM++. Chaque partition construit indépendamment un coreset local, puis ces résumés sont fusionnés pour produire le coreset global.

Cette organisation limite les échanges de données : les observations originales restent dans leurs partitions pendant la construction locale et seuls les résumés doivent être combinés lors de la phase de réduction.

La qualité observée lors des premières expériences montre que cette décomposition n'entraîne pas, sur les configurations testées, de dégradation importante par rapport à la version séquentielle.

Une fusion arborescente adaptée au modèle Spark. L'utilisation de `treeReduce` permet d'organiser la fusion des `BucketManager` sur plusieurs niveaux au lieu de centraliser immédiatement toutes les opérations de combinaison sur le driver.

Ce choix répond à une limite identifiée dans l'implémentation Flink de référence, où la fusion finale des coreset constitue une étape peu parallélisable. L'architecture Spark permet ici d'exprimer cette réduction sous une forme plus distribuée.

Il reste cependant nécessaire de distinguer l'intérêt architectural de ce choix de son impact réel sur les performances. Celui-ci dépend notamment du nombre de partitions, de la taille des coreset, du coût des opérations de fusion et des ressources matérielles disponibles. Il doit donc être évalué expérimentalement plutôt que supposé a priori.

Une évaluation distribuée de la qualité. Le calcul de la SSE est également distribué. Les centres, dont la taille reste faible par rapport au jeu de données, sont diffusés aux exécuteurs avec une variable `broadcast`. Chaque partition calcule ensuite sa contribution au coût global avant l'agrégation finale.

Cette séparation entre construction des centres et évaluation sur les données originales est importante. Elle évite de mesurer uniquement la qualité du clustering sur le coreset et permet de comparer les différentes méthodes sur une métrique commune.

Un portage qui exploite les mécanismes propres à Spark. Le passage de Flink à Spark ne se limite pas à remplacer des opérateurs par leurs équivalents syntaxiques. Plusieurs étapes du pipeline ont dû être réorganisées selon le modèle d'exécution de Spark.

Structured Streaming fournit par exemple explicitement les frontières des micro-batches avec `foreachBatch`, tandis que les opérations de `broadcast` et de `treeReduce` offrent des mécanismes directs pour diffuser les centres et fusionner les résultats intermédiaires.

Le portage a donc conduit à conserver la logique algorithmique de StreamKM++ tout en adaptant l'orchestration au modèle d'exécution du framework cible.

6.2 Limites de l'implémentation

Le clustering final reste exécuté sur le driver. Après la construction du coreset global, `KMeans.fit` est exécuté sur le driver. Cette étape reste donc séquentielle dans l'implémentation actuelle.

Ce choix est cohérent avec le principe de StreamKM++ : le coreset étant sensiblement plus petit que les données originales, l'étape finale doit rester d'un coût limité. Elle peut néanmoins devenir significative lorsque m , k ou le nombre de redémarrages augmentent.

L'ajout de ressources Spark n'accélère pas directement cette partie du traitement. Elle constitue donc une limite potentielle de la scalabilité globale.

La version Structured Streaming repose sur un état global côté driver. Dans l'adaptation streaming, chaque micro-batch produit d'abord un coreset distribué, puis ce résumé est intégré dans un `BucketManager` global conservé sur le driver.

Cette solution simplifie la gestion de l'état entre les micro-batches, mais elle introduit également un point central dans l'architecture.

Par ailleurs, l'insertion successive de coreset déjà pondérés dans cet état global n'est pas strictement équivalente à l'insertion directe de tous les points bruts dans une unique structure

merge-and-reduce. La version streaming doit donc être considérée comme une adaptation pratique du principe général, et non comme une reproduction strictement identique du traitement séquentiel.

Latence liée au traitement par micro-batches. Dans notre implémentation, Structured Streaming est utilisé en mode micro-batch. Les observations sont donc traitées lorsque Spark exécute le lot auquel elles appartiennent, et non nécessairement dès leur arrivée.

Cette organisation introduit une latence liée à la durée du micro-batch et au temps nécessaire à son traitement. Elle distingue la solution Spark d'un traitement événement par événement tel que celui utilisé dans l'architecture Flink de référence.

Cette limite est surtout importante pour des applications nécessitant une réaction à très faible latence. Elle est moins critique lorsque l'objectif principal est de maintenir périodiquement un résumé du flux.

Absence d'équivalent direct à Queryable State. La thèse exploite les mécanismes d'état de Flink pour permettre l'interrogation de certaines informations internes du job.

L'architecture Spark retenue dans ce projet ne fournit pas de mécanisme strictement équivalent permettant d'interroger directement le `BucketManager` global depuis l'extérieur du traitement.

Les modèles obtenus peuvent être transmis à un sink ou à une fonction externe, mais cela constitue un mécanisme différent de l'exposition directe de l'état interne proposée par Flink.

Sensibilité au partitionnement et à l'aléatoire. La construction distribuée dépend du nombre de partitions. Chaque partition construit son propre coreset à partir d'un sous-ensemble différent des données et utilise une graine dérivée de son indice.

Deux exécutions réalisées avec le même partitionnement et les mêmes graines peuvent être reproduites. En revanche, modifier le nombre de partitions modifie la décomposition des données et peut donc conduire à un coreset global différent.

Il ne s'agit pas nécessairement d'une erreur : les algorithmes de construction des coresets et k-means++ comportent eux-mêmes des étapes aléatoires. Cette propriété impose cependant de contrôler explicitement le partitionnement et les graines dans le protocole expérimental.

L'influence de la taille du coreset n'est pas strictement monotone. Les expériences réalisées sur le sous-ensemble de Covertype montrent une amélioration du coût lorsque m augmente dans certaines configurations, mais cette tendance n'est pas strictement monotone.

Ce résultat rappelle qu'une augmentation de la taille du coreset ne garantit pas, pour une exécution donnée, une diminution systématique de la SSE. Le processus de construction du résumé ainsi que l'initialisation k-means++ restent aléatoires.

Une caractérisation plus robuste de l'effet de m nécessiterait plusieurs répétitions avec différentes graines, puis une analyse des moyennes et des variations observées.

Absence de standardisation des variables de Covertype. Les expériences utilisent les 54 variables de Covertype sans transformation de leur échelle, afin de rester cohérentes avec le protocole retenu pour ce projet.

Certaines variables numériques peuvent prendre des valeurs de plusieurs milliers, tandis que d'autres correspondent à des indicateurs binaires. Dans une distance euclidienne au carré, les variables de grande amplitude peuvent donc contribuer davantage au coût.

Une standardisation modifierait la géométrie du problème et pourrait produire des clusters différents. Elle constitue une piste d'étude intéressante, mais n'a pas été intégrée au protocole actuel afin de conserver les mêmes données d'entrée pour l'ensemble des méthodes comparées.

Absence de mécanisme d’oubli. Le merge-and-reduce utilisé par StreamKM++ représente l’ensemble des observations vues depuis le début du flux. Les données anciennes continuent donc à contribuer au coresé même lorsque la distribution du flux évolue.

Cette propriété convient lorsque la distribution des données est relativement stable. Elle devient en revanche une limite dans un contexte de *concept drift*, où les observations récentes devraient éventuellement avoir davantage d’influence que les anciennes.

L’ajout d’une fenêtre glissante ou d’un mécanisme de décroissance des poids constituerait une extension possible, mais sort du périmètre de l’implémentation étudiée.

6.3 Bilan

Les résultats obtenus montrent que le portage de StreamKM++ vers Spark est possible sans modifier profondément les composants algorithmiques de l’approche. La construction de coresets locaux par partition et leur fusion permettent de conserver une qualité de clustering proche de la version séquentielle sur les configurations étudiées.

L’intérêt principal du portage réside dans la possibilité de distribuer les étapes les plus coûteuses sur les données originales tout en conservant un résumé global de taille contrôlée.

La solution reste toutefois partiellement centralisée : le clustering final et l’état utilisé entre les micro-batches résident sur le driver. Les performances globales dépendent également du partitionnement, du paramètre m et des coûts propres au moteur Spark.

Ces éléments montrent que l’intérêt de StreamKM++ sur Spark repose sur un compromis entre réduction des données, qualité du clustering et coût de la distribution. Les perspectives présentées dans le chapitre suivant découlent directement de ces limites.

Chapitre 7

Conclusion et perspectives

Ce rapport a décrit le portage de StreamKM++ depuis l’implémentation Flink de Bitsakis (2018) vers Apache Spark. Les trois briques algorithmiques de la thèse — Lloyd pondéré, seeding k -means++, coreset tree et merge-and-reduce — ont été implémentées et validées indépendamment de tout moteur de calcul distribué (chapitre 2), avant d’être orchestrées par une couche Spark qui ne contient elle-même aucune logique algorithmique propre (chapitre 4). Cet ordre — théorie, puis framework, puis implémentation opérateur par opérateur, puis validation de la qualité avant la scalabilité — a été tenu tout au long du projet, et a permis de découvrir et corriger un bug réel (une graine aléatoire mal propagée dans la fusion de coresets) avant qu’il n’affecte les expériences du chapitre 5.

Le portage confirme ce que l’énoncé posait comme question implicite : une partie du dataflow de la thèse se traduit directement (voire se simplifie, comme la disparition de `HandleWatermark` ou la méthodologie d’évaluation du coût), une partie se traduit avec un gain net (la fusion arborescente des coresets, qui répond directement au goulot que la thèse identifie elle-même), et une partie ne se traduit pas du tout (Queryable State) — sans que cela remette en cause la viabilité du portage dans son ensemble (chapitre 6).

7.1 Travaux futurs

Variante à requêtes périodiques (§4.3 de la thèse). Décrite mais non implémentée faute de temps (chapitre 4, §4.3) — le chemin d’implémentation est cependant déjà balisé : `Trigger.ProcessingTime` pour la ré-évaluation périodique, `BucketManager.extractCoreset` (qui ne vide jamais l’état) pour dupliquer le coreset courant à plusieurs valeurs de k .

Un vrai équivalent de Queryable State. La seule perte de fonctionnalité nette identifiée dans ce portage (chapitre 6). Une piste non explorée : un service externe léger interrogeant directement l’état exposé par `onModel`, plutôt qu’un sink `memory + SQL` — à évaluer en termes de fraîcheur et de complexité ajoutée.

Gestion du concept drift. Ajouter un mécanisme d’oubli (fenêtre glissante sur les buckets les plus anciens, ou facteur de décroissance à la `StreamingKMeans`) permettrait de traiter des flux dont la distribution change réellement dans le temps — hors du périmètre de StreamKM++ tel que défini par la thèse, mais une extension naturelle.

Poursuivre l’optimisation du kernel local. Le travail Structure-of-Arrays / hors-tas engagé sur `core/coreset` (chapitre 5, benchmarks JMH) ouvre la voie à des optimisations supplémentaires une fois les résultats consolidés — en particulier sur `CoresetTree.build`, identifié dès la spécification initiale comme le point chaud principal du pipeline.

Annexe A

Code source complet

Cette annexe inclut, via `\lstinputlisting`, le contenu **réel** des fichiers du dépôt `StreamKMpp` au moment de la rédaction de ce rapport (dossier frère `../StreamKMpp` sur la même machine). Aucun extrait n'est retapé ou reconstitué de mémoire.

A.0.1 Configuration du projet

```
1 // Scala 2.12 est imposé par Spark 3.5 (E4). On s'y tient dès maintenant pour éviter
2 // une migration au moment d'ajouter la couche Spark.
3 ThisBuild / scalaVersion := "2.12.18"
4 ThisBuild / organization := "streamkm"
5 ThisBuild / version      := "0.1.0-SNAPSHOT"
6
7 // Spark 3.5.x : dernière branche 3.5 supportant nativement Java 17 (E4).
8 // Scope "provided" : disponible en compile/test, absent du jar assemblé - c'est
9 // spark-submit qui fournit ces classes au runtime en production.
10 val sparkVersion = "3.5.3"
11
12 lazy val root = (project in file("."))
13   .settings(
14     name := "streamkm-spark",
15     libraryDependencies ++= Seq(
16       "org.scalatest" %% "scalatest" % "3.2.18" % Test,
17       "org.apache.spark" %% "spark-core" % sparkVersion % Provided,
18       "org.apache.spark" %% "spark-sql" % sparkVersion % Provided,
19       // Ajouté pour streamkm.experiments.QualityMetrics : sert de référence externe
20       // (org.apache.spark.ml.clustering.KMeans, entraîné sur le dataset complet, pas
21       // sur un coreset) dans l'expérience de comparaison à MLlib.
22       "org.apache.spark" %% "spark-mllib" % sparkVersion % Provided
23     ),
24     scalacOptions ++= Seq(
25       "-deprecation",
26       "-feature",
27       "-unchecked",
28       "-Xlint"
29     ),
30     // -ea active les 'assert' d'invariants du BucketManager pendant les tests.
31     // En production (E4) on laisse -ea désactivé : les assertions coûtent cher
32     // dans les boucles chaudes.
33     //
34     // Les --add-opens sont indispensables pour faire tourner Spark sous Java 17 :
35     // Spark/Hadoop utilisent de la réflexion sur des modules internes du JDK
36     // (java.lang, java.nio, sun.nio.ch...) que le module system de Java 17 bloque
37     // par défaut (InaccessibleObjectException sans ce flag). Ce n'est PAS une
38     // fantaisie de config : sans ça, 'sbt test' plante dès la création du
39     // SparkContext. Spark lui-même ajoute ces mêmes flags dans ses scripts de
40     // lancement (spark-submit) à partir de la 3.3.
41     Test / fork := true,
42     Test / javaOptions ++= Seq(
43       "-Xmx2g",
44       "-ea",
45       "--add-opens=java.base/java.lang=ALL-UNNAMED",
46       "--add-opens=java.base/java.lang.invoke=ALL-UNNAMED",
```

```

47     "--add-opens=java.base/java.lang.reflect=ALL-UNNAMED",
48     "--add-opens=java.base/java.io=ALL-UNNAMED",
49     "--add-opens=java.base/java.net=ALL-UNNAMED",
50     "--add-opens=java.base/java.nio=ALL-UNNAMED",
51     "--add-opens=java.base/java.util=ALL-UNNAMED",
52     "--add-opens=java.base/java.util.concurrent=ALL-UNNAMED",
53     "--add-opens=java.base/java.util.concurrent.atomic=ALL-UNNAMED",
54     "--add-opens=java.base/sun.nio.ch=ALL-UNNAMED",
55     "--add-opens=java.base/sun.nio.cs=ALL-UNNAMED",
56     "--add-opens=java.base/sun.security.action=ALL-UNNAMED",
57     "--add-opens=java.base/sun.util.calendar=ALL-UNNAMED"
58 ),
59
60 // 'run'/'runMain' : mêmes flags JVM que Test (Spark sous Java 17), avec plus de
61 // mémoire (le harnais d'expériences streamkm.experiments charge Covertypes en
62 // entier, ~580k points, en mémoire locale pour la référence séquentielle).
63 run / fork := true,
64 run / javaOptions := (Test / javaOptions).value.map {
65   case "-Xmx2g" => "-Xmx4g"
66   case other    => other
67 },
68
69 // Limitation sbt connue (cf. CLAUDE.md, note sous "Commandes") : par défaut,
70 // 'Compile / run' ET 'Compile / runMain' n'exposent PAS les dépendances
71 // "Provided" (Spark) sur leur classpath runtime - seul 'Test / *' les voit.
72 // 'run' et 'runMain' sont deux tâches sbt DISTINCTES (découvert le
73 // 2026-08-22 : corriger seulement 'run' ne suffit pas pour
74 // 'sbt runMain ...') ; les deux sont donc réalignées sur
75 // 'Compile / fullClasspath' (qui inclut Provided), pour que 'sbt run' ET
76 // 'sbt runMain streamkm.experiments.QualityMetrics' fonctionnent directement
77 // sans devoir passer par 'Test/runMain'.
78 Compile / run := Defaults.runTask(
79   Compile / fullClasspath,
80   Compile / run / mainClass,
81   Compile / run / runner
82 ).evaluated,
83 Compile / runMain := Defaults.runMainTask(
84   Compile / fullClasspath,
85   Compile / run / runner
86 ).evaluated
87 )

```

Listing A.1 – build.sbt

A.0.2 Couche core : structures de données et k-means

```

1 package streamkm.core
2
3 /**
4  * Un point pondéré de l'espace euclidien de dimension d.
5  *
6  * 'weight' vaut 1.0 pour un point brut du flux, et la masse totale du sous-
   ensemble
7  * représenté pour un point issu d'un coreset (thèse §2.3.3 : « we denote the
   weight of
8  * each  $q_i$  as the total number of points in each subset  $P$  »).
9  *
10 * ATTENTION : 'coords' n'est pas copié défensivement, ni à la construction ni à
   la
11 * lecture. C'est un choix délibéré - copier à chaque accès tuerait les
   performances dans
12 * les boucles de 'Distance' et de 'CoresetTree'. La contrepartie est un
   invariant à
13 * respecter partout : **un tableau de coordonnées n'est jamais muté après
   construction**.
14 * Toute mutation potentielle passe par '.clone()' explicite.
15 *

```

```

16  * Cette classe reste volontairement une classe « objet » (Array of Structures)
    en E1/E2.
17  * L'étape E6 introduira une représentation Structure of Arrays derrière la même
    API
18  * publique, et JMH mesurera l'écart.
19  */
20  final class Point(val coords: Array[Double], val weight: Double) extends
    Serializable {
21
22      def dim: Int = coords.length
23
24      /** Utile uniquement pour les messages d'erreur et le débogage. */
25      override def toString: String =
26          coords.mkString("Point([", ", ", "], w=$weight)")
27  }
28
29  object Point {
30
31      /** Point brut du flux : poids 1. */
32      def apply(coords: Array[Double]): Point = new Point(coords, 1.0)
33
34      def apply(coords: Array[Double], weight: Double): Point = new Point(coords,
          weight)
35
36      /** Confort pour les tests : 'Point(1.0, 2.0)'. */
37      def of(coords: Double*): Point = new Point(coords.toArray, 1.0)
38
39      /**
40       * Parsing d'une ligne CSV/SSV en Point.
41       * Renvoie None si la ligne est malformée ou n'a pas la dimension attendue - c
         'est
42       * l'équivalent du 'FlatMapToPoint' de la thèse (§4.1), qui utilise un flatMap
43       * précisément pour pouvoir rejeter les lignes invalides sans faire échouer le
         job.
44       */
45      def parse(line: String, dim: Int, sep: Char = ','): Option[Point] = {
46          val parts = line.trim.split(sep)
47          if (parts.length != dim) None
48          else {
49              val arr = new Array[Double](dim)
50              var i = 0
51              var ok = true
52              while (i < dim && ok) {
53                  try arr(i) = java.lang.Double.parseDouble(parts(i))
54                  catch { case _: NumberFormatException => ok = false }
55                  i += 1
56              }
57              if (ok) Some(new Point(arr, 1.0)) else None
58          }
59      }
60  }
61
62  object Distance {
63
64      /**
65       * Distance euclidienne AU CARRÉ.
66       *
67       * Toute la chaîne StreamKM++ (seeding  $D^2$ , coût du coresets tree, fonction
         objectif
68       * k-means) travaille sur des distances au carré. On ne calcule donc jamais de
         racine :
69       * elle serait à la fois inutile et coûteuse. C'est le point le plus chaud du
         programme.

```

```

70  */
71  def sqdist(a: Array[Double], b: Array[Double]): Double = {
72      var s = 0.0
73      var i = 0
74      val n = a.length
75      while (i < n) {
76          val d = a(i) - b(i)
77          s += d * d
78          i += 1
79      }
80      s
81  }
82
83  /**
84   * Indice du centre le plus proche et distance au carré correspondante, en un
85   * seul
86   * parcours.
87   * Le tuple alloue (et boxe le Int et le Double). C'est assumé en E1 : la
88   * lisibilité
89   * prime tant que la correction n'est pas établie. E6 remplacera cet appel par
90   * une
91   * boucle inline ou un encodage sans allocation, et JMH quantifiera le gain.
92   */
93  def nearest(p: Array[Double], centers: Array[Array[Double]]): (Int, Double) =
94  {
95      var best = 0
96      var bestD = Double.MaxValue
97      var c = 0
98      while (c < centers.length) {
99          val d = sqdist(p, centers(c))
100          if (d < bestD) { bestD = d; best = c }
101          c += 1
102      }
103      (best, bestD)
104  }

```

Listing A.2 – streamkm/core/Point.scala

```

1  package streamkm.core
2
3  import scala.util.Random
4
5  /** Résultat d'un clustering : les centres, la SSE pondérée atteinte, le nombre
6   * d'itérations. */
7  final case class KMeansModel(
8      centers: Array[Array[Double]],
9      cost: Double,
10     iterations: Int
11 ) extends Serializable {
12
13     def k: Int = centers.length
14
15     override def toString: String =
16         s"KMeansModel(k=$k, cost=$cost, iterations=$iterations)"
17 }
18
19 /**
20  * Échantillonnage pondéré, factorisé ici parce qu'il sert à trois endroits :
21  * le premier centre de k-means++, le premier représentant du coresets tree, et
22  * l'échantillonnage  $D^2$  à l'intérieur d'une feuille.
23  */
24 private[streamkm] object Sampling {

```

```

24
25 /**
26  * Tire un indice dans [0, n) avec une probabilité proportionnelle à 'w(i)'.
27  * Si la masse totale est nulle (ou négative par erreur d'arrondi), retombe
    sur
28  * un tirage uniforme plutôt que de lever une exception : ce cas se produit
29  * légitimement quand tous les points d'une feuille sont confondus.
30  */
31 def byWeight(n: Int, w: Int => Double, rng: Random): Int = {
32   var total = 0.0
33   var i      = 0
34   while (i < n) { total += w(i); i += 1 }
35
36   if (!(total > 0.0)) return rng.nextInt(n)
37
38   val target = rng.nextDouble() * total
39   var acc     = 0.0
40   i = 0
41   while (i < n) {
42     acc += w(i)
43     if (acc >= target) return i
44     i += 1
45   }
46   // Atteignable uniquement par accumulation d'erreurs flottantes.
47   n - 1
48 }
49 }
50
51 object KMeans {
52
53   /**
54    * Procédure de seeding de k-means++ (thèse, algorithme 2.2, lignes 1-3), **
    * version pondérée**.
55    *
56    * Différence assumée avec la thèse : la ligne 1 de l'algorithme 2.2 choisit
    * le premier
57    * centre « uniformly at random ». On tire ici proportionnellement au poids.
    * Sur des points
58    * bruts (tous de poids 1) les deux coïncident exactement ; sur un coresé (
    * poids très
59    * hétérogènes) le tirage uniforme biaiserait l'initialisation vers les ré
    * gions peu denses.
60    * Or dans StreamKM++, k-means++ est TOUJOURS appliqué à un coresé (§2.3.4, é
    * tape 2) -
61    * la version pondérée est donc la bonne. À mentionner dans le rapport.
62    *
63    * Coût :  $O(n \cdot k \cdot d)$ .
64    */
65   def seedPlusPlus(points: Array[Point], k: Int, rng: Random): Array[Array[
    Double]] = {
66     require(points.nonEmpty, "seedPlusPlus: ensemble de points vide")
67     require(k > 0, s"seedPlusPlus: k doit être > 0 (reçu $k)")
68
69     val n      = points.length
70     val kEff   = math.min(k, n)
71     val centers = new Array[Array[Double]](kEff)
72
73     centers(0) = points(Sampling.byWeight(n, i => points(i).weight, rng)).coords
    .clone()
74
75     // closest(i) = distance au carré de points(i) au plus proche centre déjà
    choisi.
76     // Mis à jour incrémentalement : c'est ce qui donne  $O(n \cdot k \cdot d)$  et non  $O(n \cdot k^2 \cdot d)$ 

```



```

    ).
77     val closest = new Array[Double](n)
78     var i       = 0
79     while (i < n) { closest(i) = Distance.sqdist(points(i).coords, centers(0));
        i += 1 }

80
81     var c = 1
82     while (c < kEff) {
83         // Tirage  $D^2$  pondéré :  $P(i) \propto w_i \cdot \text{closest}(i)$ 
84         val idx = Sampling.byWeight(n, j => points(j).weight * closest(j), rng)
85         centers(c) = points(idx).coords.clone()
86
87         i = 0
88         while (i < n) {
89             val d = Distance.sqdist(points(i).coords, centers(c))
90             if (d < closest(i)) closest(i) = d
91             i += 1
92         }
93         c += 1
94     }
95     centers
96 }

97
98 /**
99  * Algorithme de Lloyd pondéré (thèse, algorithme 2.1).
100  *
101  * Le centre d'un cluster est la moyenne PONDÉRÉE de ses points :
102  *  $\mu_c = \sum_{\{p \in c\}} w_p \cdot p / \sum_{\{p \in c\}} w_p$ 
103  * Oublier les poids ici est le bug classique de StreamKM++ : le code tourne,
104  *   ne plante
105  *   jamais, et produit des centres silencieusement faux dès qu'on l'applique à
106  *   un coresets.
107  *
108  * Clusters vides : on conserve le centre précédent. C'est le choix le plus
109  *   simple et il
110  *   ne dégrade pas le coût (un cluster vide contribue 0). Alternative classique
111  *   - ré-amorcer
112  *   sur le point le plus éloigné - écartée : elle rend le coût non monotone et
113  *   casse
114  *   l'assertion de convergence.
115  *
116  * Arrêt : décroissance relative du coût inférieure à 'tol', ou 'maxIter'
117  *   atteint.
118  */
119 def lloyd(
120     points: Array[Point],
121     init:   Array[Array[Double]],
122     maxIter: Int = 100,
123     tol:     Double = 1e-6
124 ): KMeansModel = {
125     require(points.nonEmpty, "lloyd: ensemble de points vide")
126     require(init.nonEmpty, "lloyd: aucun centre initial")
127
128     val k      = init.length
129     val d      = points(0).dim
130     val centers = Array.tabulate(k)(i => init(i).clone())
131
132     val sums    = Array.ofDim[Double](k, d)
133     val masses  = new Array[Double](k)
134
135     var prevCost = Double.MaxValue
136     var it       = 0
137     var done     = false

```

```

132 while (!done && it < maxIter) {
133     var c = 0
134     while (c < k) {
135         java.util.Arrays.fill(sums(c), 0.0)
136         masses(c) = 0.0
137         c += 1
138     }
139
140     // Étape d'affectation + accumulation des sommes pondérées, en un seul
141     // parcours.
142     // 'currCost' est la SSE des centres COURANTS (avant mise à jour) : c'est
143     // bien la
144     // quantité dont la décroissance prouve la convergence de Lloyd.
145     var currCost = 0.0
146     var i = 0
147     while (i < points.length) {
148         val p = points(i)
149         val (idx, dd) = Distance.nearest(p.coords, centers)
150         currCost += p.weight * dd
151         val s = sums(idx)
152         val w = p.weight
153         var j = 0
154         while (j < d) { s(j) += w * p.coords(j); j += 1 }
155         masses(idx) += w
156         i += 1
157     }
158
159     // Étape de mise à jour des centres.
160     c = 0
161     while (c < k) {
162         if (masses(c) > 0.0) {
163             val s = sums(c)
164             val m = masses(c)
165             val ctr = centers(c)
166             var j = 0
167             while (j < d) { ctr(j) = s(j) / m; j += 1 }
168         }
169         c += 1
170     }
171
172     it += 1
173     // Invariant de Lloyd : le coût ne remonte jamais.
174     assert(currCost <= prevCost + 1e-9 * math.abs(prevCost),
175            s"Lloyd: coût non monotone ($prevCost -> $currCost) - bug d'
176            affectation ou de pondération")
177
178     if (prevCost - currCost <= tol * math.max(math.abs(prevCost), 1e-300))
179         done = true
180     prevCost = currCost
181 }
182
183 // Recalcul final : 'prevCost' correspond aux centres d'AVANT la dernière
184 // mise à jour.
185 KMeansModel(centers, cost(points, centers), it)
186
187 /**
188  * k-means++ complet : 'nRestarts' exécutions indépendantes (seeding + Lloyd),
189  * on garde
190  * la moins coûteuse. La thèse et le papier original utilisent 5 (§2.3.4, é
191  * tape 2), mais
192  * le §5.3.1 montre qu'une seule application suffit en pratique - c'est un

```

```

188     paramètre
189     * d'expérience (E3), pas une constante.
190     */
191     def fit(
192         points:    Array[Point],
193         k:         Int,
194         nRestarts: Int    = 5,
195         maxIter:   Int    = 100,
196         tol:       Double = 1e-6,
197         seed:      Long   = 42L
198     ): KMeansModel = {
199         require(nRestarts > 0, s"fit: nRestarts doit être > 0 (reçu $nRestarts)")
200         val rng = new Random(seed)
201         var best: KMeansModel = null
202         var r = 0
203         while (r < nRestarts) {
204             val model = lloyd(points, seedPlusPlus(points, k, rng), maxIter, tol)
205             if (best == null || model.cost < best.cost) best = model
206             r += 1
207         }
208         best
209     }
210
211     /**
212     * Fonction objectif : SSE pondérée,  $\sum_p w_p * \min_c ||p - c||^2$ .
213     * C'est la mesure de qualité unique de tout le projet (thèse §4.2) : tous les
214     * chapitres expérimentaux comparent des valeurs produites par cette fonction.
215     */
216     def cost(points: Array[Point], centers: Array[Array[Double]]): Double = {
217         var s = 0.0
218         var i = 0
219         while (i < points.length) {
220             val p = points(i)
221             s += p.weight * Distance.nearest(p.coords, centers)._2
222             i += 1
223         }
224         s
225     }

```

Listing A.3 – streamkm/core/KMeans.scala

A.0.3 Couche coreset : coreset tree et merge-and-reduce

```

1 package streamkm.coreset
2
3 import scala.collection.mutable.ArrayBuffer
4 import scala.util.Random
5
6 import streamkm.core.{Distance, Point, Sampling}
7
8 /**
9  * Nœud de l'arbre de coreset.
10  *
11  * NOTE DE NOMMAGE - importante, à reprendre dans le rapport.
12  * La thèse appelle 'weight(v)' l'attribut servant à la descente dans l'arbre,
13  * et le définit
14  * pour une feuille comme 'cost(P_v, q_v)', c'est-à-dire une SOMME DE DISTANCES
15  * AU CARRÉ.
16  * Mais elle appelle aussi « weight » le poids des points du coreset produit,
17  * qui vaut le

```

```

15  * NOMBRE (la masse) de points représentés. Ce sont deux quantités totalement
16    différentes
17  * portant le même nom. On les sépare explicitement :
18  *   - 'cost' :  $\sum_{p \in P_v} w_p * ||p - q_v||^2$ 
19  *               sert à la descente
20  *
21  *   - 'mass' :  $\sum_{p \in P_v} w_p$ 
22  *               sert au poids du point de coreset en sortie
23  *
24  * @param points feuille : les points du cluster. Nœud interne : null.
25  * @param repr   point représentatif  $q_v$  (stocké aussi sur les nœuds internes,
26  *               il devient
27  *               le représentant du fils gauche lors d'une scission).
28  */
29 private[coreset] final class Node(
30     var points: Array[Point],
31     var repr:   Array[Double],
32     var cost:   Double,
33     var mass:   Double,
34     var left:   Node,
35     var right:  Node
36 ) {
37     def isLeaf: Boolean = left == null
38 }
39 /**
40  * Construction du coreset par arbre binaire de clustering divisif hiérarchique
41  * (thèse §2.3.3 « Coreset Tree » et algorithme 2.4).
42  *
43  * Complexité :  $O(m \cdot n)$  dans le pire cas, mais  $O(n \cdot \log m)$  en pratique, contre
44  *  $O(n \cdot m)$ 
45  * pour l'AdaptiveCoreset de l'algorithme 2.3 - c'est toute la raison d'être de
46  * cette
47  * structure.
48  */
49 object CoresetTree {
50     /**
51     * Construit un coreset d'au plus 'm' points pondérés à partir de 'points'.
52     *
53     * GARANTIE STRUCTURELLE : la somme des poids de sortie est exactement égale à
54     * la somme
55     * des poids d'entrée. C'est l'invariant que testent en priorité les tests de
56     * E2 - il
57     * attrape la grande majorité des erreurs d'implémentation de cette structure.
58     *
59     * Si 'points.length <= m', l'ensemble est déjà son propre coreset : on le
60     * renvoie tel quel.
61     */
62     def build(points: Array[Point], m: Int, rng: Random): Array[Point] = {
63         require(m > 0, s"build: m doit être > 0 (reçu $m)")
64         if (points.length <= m) return points.clone()
65
66         // --- Racine : premier représentant tiré proportionnellement au poids ---
67         // (Thèse algo 2.4 ligne 1 : « uniformly at random ». Même remarque que pour
68         // KMeans.seedPlusPlus : sur un ensemble pondéré, le tirage proportionnel à
69         // la masse
70         // est la généralisation correcte, et les deux coïncident sur des points de
71         // poids 1.)
72         val i0 = Sampling.byWeight(points.length, i => points(i).weight, rng)
73         val root = new Node(points, points(i0).coords.clone(), 0.0, 0.0, null, null)
74         recomputeLeaf(root)
75     }
76 }

```

```

69     var leaves = 1
70     val path    = new ArrayBuffer[Node](32)
71
72     // Condition d'arrêt supplémentaire par rapport à l'algorithme 2.4 : 'root.
73     // cost > 0'.
74     // Si le coût global est nul, tous les points sont confondus avec leur repré
75     // sentant :
76     // aucune scission ne peut plus rien améliorer, et l'échantillonnage  $D^2$ 
77     // serait dégénéré.
78     // Ce cas arrive réellement sur des données discrètes très redondantes.
79     while (leaves < m && root.cost > 0.0) {
80         path.clear()
81         val leaf = descend(root, rng, path)
82         val q    = sampleD2(leaf, rng)
83
84         if (q == null) return collectLeaves(root) // feuille de coût nul : sécurit
85         if (!split(leaf, q)) return collectLeaves(root) // scission dégénérée : sé
86         curité
87
88         // Remontée de la mise à jour jusqu'à la racine (algo 2.4 ligne 9).
89         // 'leaf' est le dernier élément du chemin et a déjà été mis à jour par '
90         // split'.
91         var i = path.length - 2
92         while (i >= 0) { recomputeInner(path(i)); i -= 1 }
93
94         leaves += 1
95     }
96
97     collectLeaves(root)
98 }
99
100 // -----
101 // Étapes internes (numérotées comme au §2.3.3)
102 // -----
103
104 /**
105  * Étape 1 : choisir une feuille avec une probabilité proportionnelle à son co
106  *ût.
107  *
108  *
109  * Le §2.3.3 étape 1 dit « with a probability proportional to cost( $P_\ell$ ,  $q_\ell$ ) »
110  * , et détaille
111  * une descente où un fils est choisi avec probabilité  $\text{cost}(\text{fils})/\text{cost}(\text{père})$ .
112  * La ligne 5 de
113  * l'algorithme 2.4 dit en revanche « according to their weights ». Les deux
114  * formulations
115  * s'accordent si l'on comprend « weight( $v$ ) » au sens de la définition donnée
116  * juste avant
117  * l'algorithme (weight d'une feuille =  $\text{cost}(P_v, q_v)$ ) - ce que confirme le
118  * papier original
119  * d'Ackermann et al. C'est donc bien le COÛT qui pilote la descente.
120  *
121  * Le raisonnement le confirme : une feuille déjà bien représentée (coût
122  * faible) ne doit
123  * plus être scindée ; l'effort de raffinement doit aller là où l'erreur est
124  * grande. Utiliser
125  * la masse à la place ferait dériver l'algorithme vers un simple é
126  * chantillonnage
127  * proportionnel à la densité, et lui ferait perdre la propriété de coresets.
128  *
129  * 'path' reçoit le chemin racine → feuille, pour la remontée.
130  */

```

```

117 private def descend(root: Node, rng: Random, path: ArrayBuffer[Node]): Node =
118     {
119     var node = root
120     path += node
121     while (!node.isLeaf) {
122         val total = node.cost
123         node =
124             if (!(total > 0.0)) node.left // sécurité : coût nul, choix arbitraire
125             else if (rng.nextDouble() * total < node.left.cost) node.left
126             else node.right
127         path += node
128     }
129     node
130 }
131 /**
132  * Étape 2 : dans la feuille, tirer le prochain représentant avec une
133  * probabilité
134  * proportionnelle à sa distance au carré au représentant courant (é
135  * chantillonnage  $D^2$ ).
136  * Renvoie null si la feuille est de coût nul.
137  */
138 private def sampleD2(leaf: Node, rng: Random): Array[Double] = {
139     if (!(leaf.cost > 0.0)) return null
140     val pts = leaf.points
141     val idx = Sampling.byWeight(
142         pts.length,
143         i => pts(i).weight * Distance.sqdist(pts(i).coords, leaf.repr),
144         rng
145     )
146     val chosen = pts(idx).coords
147     // Si le tirage retombe sur le représentant lui-même (distance nulle), la
148     // scission
149     // produirait un fils vide. Ne peut arriver que par erreur d'arrondi ; on
150     // refuse.
151     if (Distance.sqdist(chosen, leaf.repr) <= 0.0) null else chosen.clone()
152 }
153 /**
154  * Étape 3 : scinder la feuille en deux fils selon la proximité à 'repr' et à
155  * 'q',
156  * puis mettre à jour les attributs du nœud devenu interne.
157  *
158  * Les deux fils sont nécessairement non vides : 'repr' est à distance 0 de
159  * lui-même
160  * (donc à gauche) et 'q' est à distance 0 de lui-même et à distance > 0 de '
161  * repr'
162  * (donc à droite). Le 'false' de retour est une sécurité contre une violation
163  * de
164  * cette propriété par erreur numérique.
165  */
166 private def split(leaf: Node, q: Array[Double]): Boolean = {
167     val pts = leaf.points
168     val left = new ArrayBuffer[Point](pts.length)
169     val right = new ArrayBuffer[Point](pts.length)
170
171     var i = 0
172     while (i < pts.length) {
173         val p = pts(i)
174         if (Distance.sqdist(p.coords, leaf.repr) <= Distance.sqdist(p.coords, q))
175             left += p
176         else right += p
177         i += 1
178     }
179 }

```

```

170     }
171
172     if (left.isEmpty || right.isEmpty) return false
173
174     val l = new Node(left.toArray, leaf.repr, 0.0, 0.0, null, null)
175     val r = new Node(right.toArray, q, 0.0, 0.0, null, null)
176     recomputeLeaf(l)
177     recomputeLeaf(r)
178
179     leaf.points = null
180     leaf.left   = l
181     leaf.right  = r
182     recomputeInner(leaf)
183     true
184 }
185
186 private def recomputeLeaf(n: Node): Unit = {
187     var cost = 0.0
188     var mass = 0.0
189     val pts  = n.points
190     var i    = 0
191     while (i < pts.length) {
192         val p = pts(i)
193         cost += p.weight * Distance.sqdist(p.coords, n.repr)
194         mass += p.weight
195         i += 1
196     }
197     n.cost = cost
198     n.mass = mass
199 }
200
201 private def recomputeInner(n: Node): Unit = {
202     n.cost = n.left.cost + n.right.cost
203     n.mass = n.left.mass + n.right.mass
204 }
205
206 /**
207  * L'union des représentants des feuilles constitue le coreset. Le poids de
208  * chacun est
209  * la masse totale des points de sa feuille - d'où la conservation de la masse
210  * totale.
211  */
212 private def collectLeaves(root: Node): Array[Point] = {
213     val out  = new ArrayBuffer[Point]()
214     val stack = new ArrayBuffer[Node]()
215     stack += root
216     while (stack.nonEmpty) {
217         val n = stack.remove(stack.length - 1)
218         if (n.isLeaf) out += new Point(n.repr, n.mass)
219         else { stack += n.left; stack += n.right }
220     }
221     out.toArray
222 }

```

Listing A.4 – streamkm/coreset/CoresetTree.scala

```

1 package streamkm.coreset
2
3 import scala.collection.mutable.ArrayBuffer
4 import scala.util.Random
5
6 import streamkm.core.Point
7

```

```

8  /**
9  * Technique « merge-and-reduce » de StreamKM++ (thèse §2.3.4, algorithme 2.5).
10 *
11 * INVARIANTS (thèse §2.3.4, propriétés 1 et 2) :
12 *   - 'bucket(0)' est le tampon d'entrée : entre 0 et m points.
13 *   - 'bucket(i)' pour  $i \geq 1$  est soit VIDE, soit exactement de taille m.
14 *   - un 'bucket(i)' plein pour  $i \geq 1$  représente exactement  $2^{(i-1)} \cdot m$  points
    du flux.
15 *   - le nombre de buckets croît en  $O(\log(n/m))$  : on n'a pas besoin de connaî
    tre n a priori,
16 *     les niveaux sont créés à la demande.
17 *
18 * Ces invariants sont vérifiés par 'checkInvariants()', appelé sous 'assert' (
    donc actif
19 * seulement avec -ea, cf. build.sbt).
20 *
21 * Cette classe est l'état que la thèse maintient dans chaque subtask parallèle
    de
22 * 'FlatMapToPartialCoresets' (§4.1). Côté Spark, c'est l'accumulateur d'un '
    treeAggregate'
23 * (E4) : 'seqOp = insert', 'combOp = merge'.
24 *
25 * NON THREAD-SAFE - un 'BucketManager' appartient à une seule partition.
26 */
27 final class BucketManager(val m: Int, val seed: Long) extends Serializable {
28     require(m > 0, s"BucketManager: m doit être > 0 (reçu $m)")
29
30     private val rng = new Random(seed)
31     private val buckets: ArrayBuffer[ArrayBuffer[Point]] = ArrayBuffer(new
        ArrayBuffer[Point](m))
32     private var seen: Double = 0.0
33     private var collapsed: Boolean = false
34
35     /** Masse totale consommée depuis la création ( $\Sigma$  des poids insérés). */
36     def numPointsSeen: Double = seen
37
38     /** Nombre de niveaux actuellement alloués ( $\approx \log_2(n/m) + 2$ ). */
39     def numLevels: Int = buckets.length
40
41     /**
42      * Insère un point dans le flux (algorithme 2.5).
43      *
44      * Déroulé, à partir de la description en prose du §2.3.4 :
45      *   - le point va toujours dans B0 ;
46      *   - si B0 est plein et B1 vide, on déplace B0 dans B1 (aucun coreset n'est
        calculé,
47      *     B1 contient m points bruts et représente donc bien  $2^0 \cdot m$  points) ;
48      *   - si B1 est plein, on construit un coreset de  $B0 \cup B1$  ( $2m$  points  $\rightarrow m$ 
        points) et on
49      *     le range dans B2 ; si B2 est plein, on recommence avec B2, et ainsi de
        suite jusqu'à
50      *     trouver un niveau libre.
51      */
52     def insert(p: Point): Unit = {
53         if (collapsed)
54             throw new IllegalStateException(
55                 "BucketManager: insertion interdite après merge(). Un manager fusionné
                    est un " +
56                 "résultat de combOp, il n'est plus alimenté par le flux."
57             )
58
59         buckets(0) += p
60         seen += p.weight

```



```

61
62     if (buckets(0).length >= m) {
63         ensureLevel(1)
64         if (buckets(1).isEmpty) {
65             // B1 vide : simple déplacement, pas de réduction.
66             buckets(1) += buckets(0)
67             buckets(0).clear()
68         } else {
69             // B1 plein : cascade de fusions-réductions.
70             var union: Array[Point] = (buckets(0) ++ buckets(1)).toArray
71             buckets(0).clear()
72             buckets(1).clear()
73
74             var level = 2
75             var placed = false
76             while (!placed) {
77                 val reduced = CoresetTree.build(union, m, rng)
78                 ensureLevel(level)
79                 if (buckets(level).isEmpty) {
80                     buckets(level) += reduced
81                     placed = true
82                 } else {
83                     union = reduced ++ buckets(level)
84                     buckets(level).clear()
85                     level += 1
86                 }
87             }
88         }
89         assert(checkInvariants(), "BucketManager: invariants violés après insert")
90     }
91 }
92
93 /** Insertion en lot. Renvoie 'this' pour pouvoir chaîner dans un 'seqOp'. */
94 def insertAll(it: Iterator[Point]): this.type = {
95     while (it.hasNext) insert(it.next())
96     this
97 }
98
99 def insertAll(pts: Array[Point]): this.type = insertAll(pts.iterator)
100
101 /**
102  * Union de tous les buckets non vides, SANS réduction.
103  *
104  * C'est le « final coreset » au sens du §2.3.4 : au plus  $m \cdot (\log_2(n/m) + 2)$ 
105  * points
106  * pondérés. C'est cet ensemble que la thèse passe à k-means++ à l'étape 2 de
107  * StreamKM++.
108  */
109 def union(): Array[Point] = {
110     val out = new ArrayBuffer[Point]()
111     var i = 0
112     while (i < buckets.length) { out += buckets(i); i += 1 }
113     out.toArray
114 }
115
116 /**
117  * Union de tous les buckets, RÉDUITE à m points par le coreset tree.
118  *
119  * C'est le « partial coreset » du §4.1 : ce que chaque subtask parallèle émet
120  * en fin de
121  * flux. Ne vide pas l'état - on peut donc interroger le clustering courant
122  * sans
123  * interrompre la consommation (c'est ce qui rendra possible la variante à

```

```

    requêtes
121 * périodiques du §4.3, étape E8).
122 */
123 def extractCoreset(): Array[Point] = CoresetTree.build(union(), m, rng)
124
125 /**
126  * Fusion de deux structures indépendantes, pour le 'combOp' d'un '
127   * treeAggregate' /
128   * 'treeReduce'.
129   *
130   * Repose sur les deux observations du §2.3.4 :
131   * 1. l'union de deux  $\varepsilon$ -coresets d'ensembles disjoints est un  $\varepsilon$ -coreset de l'
132     * union ;
133   * 2. un  $\varepsilon$ -coreset d'un  $\delta$ -coreset est un  $(\varepsilon + \delta + \varepsilon\delta)$ -coreset.
134   * La seconde explique que l'erreur s'accumule à chaque niveau de fusion - c'
135     * est
136     * précisément pourquoi 'streamkm.spark.StreamKMPlusPlus.mergeCoresets'
137     * utilise un
138     * 'treeReduce' de profondeur bornée plutôt qu'une réduction linéaire : la
139     * profondeur de
140     * l'arbre de fusion borne le nombre d'empilements. Voir CLAUDE.md, décision
141     * 11,
142     * pour le calcul explicite de cette borne.
143     *
144     * La réduction 'CoresetTree.build(all, m, out.rng)' utilise le générateur de
145     * 'out' -
146     * celui construit avec la graine combinée 'seed ^ other.seed' - et NON celui
147     * de 'this'
148     * ni celui de 'other'. C'est important : à chaque niveau d'un 'treeReduce',
149     * ce générateur
150     * combine la graine des DEUX opérandes de ce niveau précis, garantissant un
151     * aléa
152     * réellement indépendant à chaque étage de l'arbre de fusion, pas seulement
153     * au premier
154     * niveau (partition → coreset partiel). Utiliser 'this.rng' ou 'other.rng' à
155     * la place
156     * ferait dépendre l'aléa de CE niveau uniquement de l'un des deux opérandes,
157     * en ignorant
158     * l'autre - un bug identifié et corrigé le 2026-08-22 (CLAUDE.md, décision
159     * 10).
160     *
161     * Le résultat est « collapsé » : son état n'est plus un empilement de buckets
162     * valide
163     * au sens des invariants, mais un unique coreset de taille m. Toute insertion
164     * ultérieure
165     * est refusée. C'est sans conséquence pour 'treeAggregate'/'treeReduce', dont
166     * le 'combOp'
167     * ne reçoit jamais que des résultats de 'seqOp'/'combOp', jamais de points
168     * bruts.
169     *
170     * Ne modifie ni 'this' ni 'other'.
171     */
172 def merge(other: BucketManager): BucketManager = {
173   require(other.m == m, s"merge: tailles de coreset incompatibles (${m} vs ${
174     other.m})")
175
176   val out = new BucketManager(m, seed ^ other.seed)
177   val all = union() ++ other.union()
178
179   out.buckets(0).clear()
180   out.ensureLevel(1)
181   out.buckets(1) += CoresetTree.build(all, m, out.rng)
182   out.seen = seen + other.seen

```

```

164     out.collapsed = true
165     out
166 }
167
168 // -----
169
170 private def ensureLevel(level: Int): Unit =
171     while (buckets.length <= level) buckets += new ArrayBuffer[Point](m)
172
173 /**
174  * Vérification des invariants du §2.3.4. Utilisée sous 'assert' : coûteuse et
175  * désactivée en production.
176  */
177 private[coreset] def checkInvariants(): Boolean = {
178     if (buckets(0).length > m) return false
179     var i = 1
180     while (i < buckets.length) {
181         val s = buckets(i).length
182         if (s != 0 && s != m) return false
183         i += 1
184     }
185     true
186 }
187
188 override def toString: String =
189     s"BucketManager(m=$m, seen=$seen, levels=${buckets.map(_.length).mkString("[", ",", "]")}" +
190     (if (collapsed) ", collapsed" else "") + ")"
191 }
192
193 object BucketManager {
194
195     /**
196     * Taille de coreset recommandée par la thèse (§2.3.4, clarification 1) et
197     * expérimentalement au §5.2.1 :  $m = 200 \cdot k$  est un bon compromis temps / qualité.
198     * C'est la valeur par défaut de nos expériences, pas une contrainte de l'
199     * algorithme.
200     */
201     def recommendedM(k: Int): Int = 200 * k
202
203     def apply(m: Int, seed: Long = 42L): BucketManager = new BucketManager(m, seed)
204 }

```

Listing A.5 – streamkm/coreset/BucketManager.scala

A.0.4 Couche spark : portage distribué

```

1 package streamkm.spark
2
3 import org.apache.spark.rdd.RDD
4 import org.apache.spark.sql.DataFrame
5 import org.apache.spark.sql.streaming.StreamingQuery
6
7 import streamkm.core.{KMeans, KMeansModel, Point}
8 import streamkm.coreset.BucketManager
9
10 /**
11  * Paramètres de StreamKM++ (SPEC §4).
12  *

```

```

13 * 'm' est la taille de coresets - la thèse recommande  $m = 200 \cdot k$  (§2.3.4,
14 * 'BucketManager.recommendedM'). 'nRestarts' est le nombre d'applications de
15 * k-means++ sur le coresets final (thèse §2.3.4 étape 2 : 5 par défaut ; le
16 * §5.3.1 montre qu'une seule suffit en pratique - paramètre d'expérience E3,
17 * pas une constante algorithmique).
18 */
19 final case class StreamKMParams(
20     k: Int,
21     m: Int,
22     nRestarts: Int = 5,
23     seed: Long = 42L
24 )
25
26 /**
27  * Couche 3 (SPEC §4) : orchestration Spark de StreamKM++, portage du dataflow
28  * Flink §4.1 de la thèse.
29  *
30  * RÈGLE D'OR (SPEC §1, rappelée ici) : ce fichier ne contient AUCUNE logique
31  * algorithmique. Tout le calcul vient de 'streamkm.core' (KMeans, déjà testé
32  * E1) et 'streamkm.coreset' (BucketManager/CoresetTree, déjà testé E2). Cette
33  * couche se contente de distribuer des appels à ces fonctions sur des
34  * partitions RDD et d'orchestrer le micro-batch Structured Streaming. Aucune
35  * modification n'a été nécessaire dans 'core'/'coreset' pour ça - voir
36  * CLAUDE.md, décision « E3-1 ».
37  */
38 object StreamKMPlusPlus {
39
40     /**
41      * Un coresets partiel par PARTITION - portage direct de l'opérateur
42      * 'FlatMapToPartialCoresets' de la thèse (§4.1) : « each one of the parallel
43      * subtasks... consumes the input points according to algorithm 2.5 ». Une
44      * partition Spark joue exactement le rôle d'un subtask Flink.
45      *
46      * Graine dérivée de l'indice de partition ('p.seed + idx') plutôt que
47      * partagée : nécessaire pour que les expériences de scalabilité (METHODO
48      * §4.0, E3bis) restent reproductibles indépendamment du nombre de
49      * partitions choisi. Avec une graine unique partagée entre partitions,
50      * changer 'numPartitions' changerait aussi la suite aléatoire consommée par
51      * chaque partition, et confondrait l'effet mesuré (le parallélisme) avec un
52      * simple changement de graine - biais qu'on évite ici.
53      */
54     def partialCoresets(rdd: RDD[Point], p: StreamKMParams): RDD[Array[Point]] =
55         rdd.mapPartitionsWithIndex { (idx, it) =>
56             val bm = new BucketManager(p.m, p.seed + idx)
57             bm.insertAll(it)
58             Iterator.single(bm.extractCoreset())
59         }
60
61     /**
62      * Coresets final unique pour tout le RDD - remplace l'opérateur
63      * 'FlatMapToFinalCoreset' de la thèse (§4.1), qui y est non-parallèle
64      * (fusion séquentielle des coresets partiels). C'est l'apport principal
65      * identifié par METHODO §3 : la thèse elle-même repère cet opérateur comme
66      * le goulot qui plafonne le speedup au-delà de 8 cœurs (§5.3.1). Ici, la
67      * fusion se fait en arbre via 'RDD.treeReduce'.
68      *
69      * Implémentation : un 'BucketManager' par partition (équivalent d'un
70      * 'seqOp' qui insère tous les points de la partition, déjà couvert par les
71      * tests E2 de 'BucketManager.insert'/'insertAll'), fusionnés deux à deux par
72      * 'BucketManager.merge' (le 'combOp', réutilisé tel quel - voir CLAUDE.md
73      * décision « E3-1 ») via 'treeReduce(f, depth)'.
74      *
75      * Pourquoi pas 'RDD.treeAggregate' directement, alors que la SPEC (§4,

```

```

76 * exemple de pipeline) l'utilise avec un 'zeroValue' unique partagé ? Parce
77 * que ce 'zeroValue' unique porte UNE SEULE graine pour toutes les
78 * partitions (Spark clone l'objet par tâche, mais avec le même état RNG
79 * initial), ce qui casse la recommandation ci-dessus de graines
80 * indépendantes par partition. 'mapPartitionsWithIndex' + 'treeReduce'
81 * produit exactement la même opération (un accumulateur par partition,
82 * fusionnés en arbre de profondeur 'depth') tout en gardant des graines
83 * distinctes. Écart documenté par rapport à l'exemple de code de la SPEC,
84 * mais pas à ses signatures ni à ses invariants.
85 */
86 def mergeCoresets(rdd: RDD[Point], p: StreamKMPParams, depth: Int = 2): Array[
    Point] = {
87     val perPartition: RDD[BucketManager] = rdd.mapPartitionsWithIndex { (idx, it
    ) =>
88         val bm = new BucketManager(p.m, p.seed + idx)
89         bm.insertAll(it)
90         Iterator.single(bm)
91     }
92     perPartition.treeReduce((a, b) => a.merge(b), depth).extractCoreset()
93 }
94
95 /**
96  * Boucle Structured Streaming complète - portage du dataflow entier §4.1 de
97  * la thèse (HdfsSource → FlatMapToPoint → FlatMapToPartialCoresets →
98  * FlatMapToFinalCoreset → FlatMapToKmeansPP → HdfsSink).
99  *
100  * Contrat d'entrée : 'source' doit exposer une colonne 'featuresCol' de type
101  * 'array<double>' (un point brut du flux, poids implicite 1.0 - le poids
102  * n'apparaît jamais dans les données d'entrée, seulement dans les coresets
103  * produits en interne, cf. 'Point.scala'). C'est l'équivalent du
104  * 'FlatMapToPoint' de la thèse, qui parse une ligne en 'Point' ; ici la
105  * conversion est un simple 'Row => Point', volontairement sans passer par un
106  * 'Encoder[Point]' Spark (Point n'est pas un case class, lui en donner un
107  * ajouterait de la complexité sans bénéfice : on repasse en RDD dès la
108  * lecture de la colonne, comme le fait 'mergeCoresets').
109  *
110  * État global : un unique 'BucketManager' maintenu côté driver entre les
111  * micro-batches (mémoire bornée par construction,  $O(m \cdot \log(n/m))$ , thèse
112  * §2.3.4). Chaque micro-batch produit son propre coreset distribué via
113  * 'mergeCoresets', qui est ensuite intégré ('insert') dans cet état global -
114  * exactement la même opération de fusion que 'FlatMapToFinalCoreset',
115  * appliquée cette fois entre micro-batches plutôt qu'entre partitions.
116  *
117  * Instrumentation : une ligne de log par micro-batch (partitions, m, k,
118  * masse vue, temps de construction du coreset, coût du modèle) - le
119  * minimum nécessaire pour l'analyse de scalabilité du ch. 5 du rapport,
120  * sans dépendance à un framework de métriques. 'numPartitions' et 'm' sont
121  * déjà des leviers explicites (le premier via 'source.repartition(N)' avant
122  * l'appel, le second via 'StreamKMPParams.m') : rien à changer ici pour les
123  * faire varier dans les expériences.
124  */
125 def run(
126     source:      DataFrame,
127     p:            StreamKMPParams,
128     onModel:      KMeansModel => Unit,
129     featuresCol:  String = "features"
130 ): StreamingQuery = {
131
132     val global = new BucketManager(p.m, p.seed)
133
134     source.writeStream.foreachBatch { (batch: DataFrame, batchId: Long) =>
135         val t0 = System.nanoTime()
136

```

```

137     val points: RDD[Point] = batch.rdd.map { row =>
138         val coords = row.getAs[Seq[Double]](featuresCol).toArray
139         Point(coords)
140     }
141     val numPartitions = points.getNumPartitions
142
143     val partial = mergeCoresets(points, p.copy(seed = p.seed + batchId))
144     partial.foreach(global.insert)
145
146     val model      = KMeans.fit(global.extractCoreset(), p.k, p.nRestarts, seed
147                               = p.seed)
148     val elapsedMs = (System.nanoTime() - t0) / 1e6
149
150     // scalastyle:off println
151     println(
152         s"[StreamKM++] batch=$batchId partitions=$numPartitions m=${p.m} k=${p.k}
153         } " +
154         f"pointsSeen=${global.numPointsSeen}%.0f buildTimeMs=$elapsedMs%.1f cost
155         =${model.cost}%.4g"
156     )
157     // scalastyle:on println
158     onModel(model)
159 }.start()

```

Listing A.6 – streamkm/spark/StreamKMPlusPlus.scala

```

1 package streamkm.spark
2
3 import org.apache.spark.rdd.RDD
4
5 import streamkm.core.{KMeans, KMeansModel, Point}
6
7 /**
8  * Portage Spark de la méthodologie d'évaluation du coût (thèse §4.2 : job séparé,
9  * indépendant du dataflow §4.1, qui recalcule 'cost(P, C) =  $\sum_x w_x \cdot D^2(x, C)$ '
10  * en
11  * parallèle sur l'ensemble du flux à partir d'un jeu de centres déjà connu).
12  *
13  * RÈGLE D'OR (SPEC §1) : aucune logique algorithmique ici. La formule du coût
14  * vit dans
15  * 'KMeans.cost' (couche 1, déjà testée par 'KMeansSpec') ; ce fichier ne fait
16  * que la
17  * distribuer sur les partitions d'un RDD et diffuser les centres par broadcast.
18  *
19  * C'est l'instrument de mesure nécessaire à TOUTES les expériences du ch. 5 (E1
20  * -
21  * correction contre une vérité terrain ou MLlib, E2 - trade-off taille de
22  * coreset, E6 -
23  * comparaison de référence) : elles comparent toutes des valeurs produites par
24  * cette
25  * fonction, jamais le coût mesuré sur le coreset lui-même (qui n'est pas
26  * comparable,
27  * cf. le commentaire de 'demo.Main').
28  *
29  * Correspondance avec la thèse (METHODO §3, table "§4.2") :
30  * - 'FlatMapToCentroid' + '.broadcast()' → 'sc.broadcast(centers)'
31  * - 'CoFlatMapToPartialCost' (connected stream) → 'mapPartitions(... KMeans
32  * .cost ...)'
33  * - 'CoFlatMapToFinalCost' (opérateur par. 1) → 'treeReduce(_ + _)'
```

```

26  * Ce que la thèse implémente avec un connected stream, un buffer d'attente et
    trois
27  * opérateurs Flink dédiés tient ici en trois lignes de 'broadcast' + '
    treeReduce'.
28  */
29  object CostEvaluator {
30
31    /**
32     * Coût total (SSE pondérée) de 'centers' sur l'ensemble du RDD 'points',
        calculé en
33     * parallèle. Chaque partition appelle 'KMeans.cost' (couche 1) sur sa propre
        part des
34     * données avec les centres broadcastés, puis les coûts partiels sont sommés
        par un
35     * 'treeReduce' (arbre de sommation, plus robuste qu'une réduction linéaire
        pour un
36     * grand nombre de partitions - même raisonnement que 'StreamKMPlusPlus.
        mergeCoresets').
37     *
38     * Contrairement à la fusion de coresets, cette agrégation est une simple
        somme :
39     * aucune composition d'erreur ici (l'addition est associative et exacte à la
        précision
40     * flottante près), donc pas de tolérance à dériver - la valeur ne dépend pas
        du nombre
41     * de partitions choisi (à l'ordre de sommation flottant près).
42     */
43     def cost(points: RDD[Point], centers: Array[Array[Double]], depth: Int = 2):
        Double = {
44         val bcCenters = points.sparkContext.broadcast(centers)
45         points
46         .mapPartitions(it => Iterator.single(KMeans.cost(it.toArray, bcCenters.
            value)))
47         .treeReduce(_ + _, depth)
48     }
49
50     /** Confort : coût d'un 'KMeansModel' déjà ajusté (celui produit par 'onModel'
        de 'run'). */
51     def cost(points: RDD[Point], model: KMeansModel): Double = cost(points, model.
        centers)
52 }

```

Listing A.7 – streamkm/spark/CostEvaluator.scala

A.0.5 Couche experiments : harnais de mesure de qualité

```

1  package streamkm.experiments
2
3  import java.io.{File, FileOutputStream, OutputStreamWriter, PrintWriter}
4  import java.nio.charset.StandardCharsets
5
6  import scala.io.Source
7  import scala.util.Try
8
9  import org.apache.spark.ml.clustering.{KMeans => MLKMeans}
10 import org.apache.spark.ml.linalg.{SQLDataTypes, Vectors}
11 import org.apache.spark.rdd.RDD
12 import org.apache.spark.sql.{DataFrame, Row => SqlRow, SparkSession}
13 import org.apache.spark.sql.types.{StructField, StructType}
14 import org.apache.spark.storage.StorageLevel
15
16 import streamkm.core.{KMeans, Point}

```

```

17 import streamkm.coreset.BucketManager
18 import streamkm.spark.{CostEvaluator, StreamKMParams, StreamKMPlusPlus}
19
20
21 /**
22  * Experiments de qualite de StreamKM++ sur Coverttype.
23  *
24  * Trois approches sont comparees :
25  *
26  *   1. StreamKM++ sequentiel ;
27  *   2. StreamKM++ distribue avec Spark ;
28  *   3. Spark MLlib KMeans comme reference externe.
29  *
30  * La mesure de qualite est toujours la SSE evaluee sur les donnees
31  * originales :
32  *
33  *    $\sum_x \min_c ||x - c||^2$ 
34  *
35  * Le coreset sert uniquement a calculer les centres.
36  */
37 object QualityMetrics {
38
39     // -----
40     // Configuration commune des experiences
41     // -----
42
43     private val Dim = 54
44
45     private val DefaultSeed = 2026L
46     private val DefaultPartitions = 8
47
48     private val NRestarts = 5
49     private val MaxIter = 100
50     private val Tol = 1e-6
51
52     // Valeurs utilisees pour l'etude de l'effet du nombre de clusters.
53     private val KValues =
54         Seq(10, 20, 30, 40, 50)
55
56     // Valeurs utilisees pour l'etude de la taille du coreset.
57     private val MValues =
58         Seq(1000, 5000, 10000, 20000)
59
60
61     final case class ResultRow(
62         dataset: String,
63         method: String,
64         k: Int,
65         m: Int,
66         run: Int,
67         seed: Long,
68         cost: Double
69     )
70
71
72     // =====
73     // Parsing de Coverttype
74     // =====
75
76     /**
77      * Parse une ligne du dataset Coverttype.
78      *
79      * Le fichier UCI contient 55 colonnes :

```



```

80      *
81      *   - 54 features ;
82      *   - 1 label de classe.
83      *
84      * Comme nous faisons du clustering non supervise, le label n'est pas utilise.
85      */
86      private def parseCovertypeline(
87      line: String,
88      dim: Int
89  ): Option[Point] = {
90
91      val columns = line.trim.split(",")
92
93      if (columns.length < dim) {
94          None
95      } else {
96          Point.parse(
97              columns.take(dim).mkString(","),
98              dim
99          )
100      }
101  }
102
103
104  // =====
105  // Lecture locale / sequentielle
106  // =====
107
108  /**
109   * Ouvre Covertypeline et fournit un Iterator[Point].
110   *
111   * Contrairement a l'ancienne implementation, l'ensemble du dataset
112   * n'est PAS charge dans un Array[Point].
113   *
114   * Les points sont lus progressivement depuis le fichier.
115   *
116   * maxPoints est principalement utilise pour les smoke tests :
117   *
118   *   Some(20000) -> premiers 20 000 points
119   *   None       -> dataset complet
120   */
121  private def withPointsFromFile[A](
122  path: String,
123  dim: Int,
124  maxPoints: Option[Int]
125  )(
126  f: Iterator[Point] => A
127  ): A = {
128
129      val source = Source.fromFile(path)
130
131      try {
132
133          val allLines = source.getLines()
134
135          val selectedLines =
136              maxPoints match {
137                  case Some(n) => allLines.take(n)
138                  case None   => allLines
139              }
140
141          val points =
142              selectedLines.flatMap { line =>

```

```

143         parseCovertypeline(line, dim)
144     }
145
146     f(points)
147
148 } finally {
149     source.close()
150 }
151 }
152
153 /**
154  * Charge une petite partie de Covertypeline dans un Array.
155  *
156  * Cette methode est conservee principalement pour les tests de validation
157  * et les petits jeux de donnees.
158  *
159  * Pour les experiences Spark sur le dataset complet, on utilise
160  * loadCovertypelineRDD afin d'eviter de charger toutes les donnees sur le driver.
161  */
162 def loadCovertypeline(
163     path: String,
164     dim: Int = Dim,
165     maxPoints: Option[Int] = None
166 ): Array[Point] = {
167
168     withPointsFromFile(
169         path,
170         dim,
171         maxPoints
172     ) { points =>
173         points.toArray
174     }
175 }
176
177 /**
178  * Calcule la SSE d'un ensemble de centres sur un fichier Covertypeline
179  * sans charger tout le dataset en memoire.
180  *
181  * Les points sont lus par blocs.
182  *
183  * On reutilise KMeans.cost afin de ne pas dupliquer la formule
184  * mathematique de la SSE.
185  */
186 private def costFromFile(
187     path: String,
188     centers: Array[Array[Double]],
189     dim: Int,
190     maxPoints: Option[Int],
191     chunkSize: Int = 4096
192 ): Double = {
193
194     withPointsFromFile(
195         path,
196         dim,
197         maxPoints
198     ) { points =>
199
200         points
201         .grouped(chunkSize)
202         .map { chunk =>
203             KMeans.cost(
204                 chunk.toArray,
205                 centers

```

```

206         )
207     }
208     .sum
209 }
210 }
211
212
213 // =====
214 // Lecture distribuee Spark
215 // =====
216
217 /**
218  * Charge directement Coverttype avec Spark.
219  *
220  * C'est la difference principale avec l'ancienne version :
221  *
222  * AVANT :
223  *
224  *     Source.fromFile
225  *       -> Array[Point]
226  *       -> sc.parallelize(...)
227  *
228  * MAINTENANT :
229  *
230  *     sc.textFile(...)
231  *       -> RDD[String]
232  *       -> RDD[Point]
233  *
234  * Les donnees ne transitent donc plus par un gros Array sur le driver.
235  */
236 def loadCoverttypeRDD(
237     spark: SparkSession,
238     path: String,
239     dim: Int = Dim,
240     numPartitions: Int = DefaultPartitions,
241     maxPoints: Option[Int] = None
242 ): RDD[Point] = {
243
244     val lines =
245         spark.sparkContext.textFile(
246             path,
247             numPartitions
248         )
249
250     /*
251      * Pour le dataset complet, aucune operation supplementaire
252      * n'est necessaire.
253      *
254      * Pour un smoke test avec maxPoints, zipWithIndex permet de
255      * conserver les N premieres lignes sans les collecter sur le driver.
256      */
257     val selectedLines =
258         maxPoints match {
259
260             case Some(n) =>
261                 lines
262                     .zipWithIndex()
263                     .filter {
264                         case (_, index) =>
265                             index < n.toLong
266                     }
267                     .keys
268

```

```

269         case None =>
270             lines
271     }
272
273     val parsed =
274         selectedLines.flatMap { line =>
275             parseCovertTypeLine(line, dim)
276         }
277
278     /*
279      * textFile utilise numPartitions comme minimum.
280      * On garantit ici explicitement le nombre de partitions utilise
281      * par notre protocole experimental.
282      *
283      * Cette repartition n'est faite qu'UNE SEULE FOIS au chargement,
284      * et non a chaque valeur de k ou m.
285      */
286     if (parsed.getNumPartitions == numPartitions) {
287         parsed
288     } else {
289         parsed.repartition(numPartitions)
290     }
291 }
292
293
294 // =====
295 // 1. StreamKM++ sequentiel
296 // =====
297
298 /**
299  * Version de reference sequentielle.
300  *
301  * Etape 1 :
302  * lecture du fichier point par point et construction du coresets.
303  *
304  * Etape 2 :
305  * k-means++ est applique au coresets.
306  *
307  * Etape 3 :
308  * le fichier est relu afin de calculer la SSE sur les donnees originales.
309  *
310  * La memoire utilisee ne depend donc pas directement du nombre total
311  * de points du dataset.
312  */
313 def sequentialCostFromFile(
314     path: String,
315     k: Int,
316     m: Int,
317     seed: Long,
318     dim: Int = Dim,
319     maxPoints: Option[Int] = None
320 ): Double = {
321
322     val manager =
323         new BucketManager(
324             m,
325             seed
326         )
327
328     // Premier passage : construction du coresets.
329     withPointsFromFile(
330         path,
331         dim,

```

```

332     maxPoints
333 ) { points =>
334
335     manager.insertAll(points)
336 }
337
338 require(
339     manager.numPointsSeen > 0.0,
340     s"No valid point found in $path"
341 )
342
343 val coreset =
344     manager.extractCoreset()
345
346 val model =
347     KMeans.fit(
348         coreset,
349         k,
350         nRestarts = NRestarts,
351         maxIter = MaxIter,
352         tol = Tol,
353         seed = seed
354     )
355
356 // Deuxieme passage : cout sur les donnees completes.
357 costFromFile(
358     path,
359     model.centers,
360     dim,
361     maxPoints
362 )
363 }
364
365
366 // =====
367 // 2. StreamKM++ distribue Spark
368 // =====
369
370 /**
371  * Version Spark de StreamKM++.
372  *
373  * Le RDD est deja partitionne et mis en cache dans main.
374  * Il n'est donc pas repartitionne a chaque experience.
375  */
376 def distributedCost(
377     points: RDD[Point],
378     k: Int,
379     m: Int,
380     seed: Long
381 ): Double = {
382
383     val params =
384         StreamKMParams(
385             k = k,
386             m = m,
387             nRestarts = NRestarts,
388             seed = seed
389         )
390
391     val coreset =
392         StreamKMPlusPlus.mergeCoresets(
393             points,
394             params

```

```

395     )
396
397     val model =
398         KMeans.fit(
399             coreset,
400             k,
401             nRestarts = NRestarts,
402             maxIter = MaxIter,
403             tol = Tol,
404             seed = seed
405         )
406
407     /*
408     * Important :
409     * on evalue les centres sur toutes les donnees du RDD,
410     * pas seulement sur le coreset.
411     */
412     CostEvaluator.cost(
413         points,
414         model.centers
415     )
416 }
417
418
419 // =====
420 // 3. Spark MLlib KMeans
421 // =====
422
423 /**
424  * Convertit une seule fois le RDD[Point] en DataFrame MLlib.
425  *
426  * Ce DataFrame sera reutilise pour toutes les valeurs de k.
427  */
428 private def toFeaturesDataFrame(
429     spark: SparkSession,
430     points: RDD[Point]
431 ): DataFrame = {
432
433     val schema =
434         StructType(
435             Seq(
436                 StructField(
437                     "features",
438                     SQLDataTypes.VectorType,
439                     nullable = false
440                 )
441             )
442         )
443
444     val rows =
445         points.map { point =>
446             SqlRow(
447                 Vectors.dense(point.coords)
448             )
449         }
450
451     spark.createDataFrame(
452         rows,
453         schema
454     )
455 }
456
457

```

```

458 /**
459  * Reference externe avec Spark MLlib.
460  *
461  * MLlib n'utilise pas notre coresets :
462  * KMeans est entraine directement sur les donnees completes.
463  *
464  * Pour rendre la comparaison plus stable, plusieurs initialisations
465  * independantes sont testees et le meilleur cout est conserve.
466  *
467  * Remarque :
468  * MLlib utilise son propre mecanisme d'initialisation.
469  * Il s'agit donc d'une reference externe de qualite et non d'une
470  * reproduction exacte de KMeans.fit.
471  */
472 def mllibCost(
473   features: DataFrame,
474   k: Int,
475   seed: Long
476 ): Double = {
477
478   (0 until NRestarts)
479     .map { restart =>
480
481     /**
482      * On espace volontairement les graines entre redemarrages
483      * afin d'eviter que deux repetitions successives utilisent
484      * presque exactement la meme serie de graines.
485      */
486     val restartSeed =
487       seed + restart.toLong * 1000003L
488
489     val model =
490       new MLKMeans()
491         .setK(k)
492         .setSeed(restartSeed)
493         .setFeaturesCol("features")
494         .setMaxIter(MaxIter)
495         .setTol(Tol)
496         .fit(features)
497
498     model.summary.trainingCost
499   }
500   .min
501 }
502
503
504 // =====
505 // Experience A : effet de k
506 // =====
507
508 /**
509  * Fait varier le nombre de clusters k.
510  *
511  * Pour chaque valeur :
512  *
513  *   - m = 200 * k ;
514  *   - StreamKM++ sequentiel ;
515  *   - StreamKM++ Spark ;
516  *   - Spark MLlib.
517  */
518 def runKSweep(
519   spark: SparkSession,
520   path: String,

```

```

521     datasetName: String,
522     pointsRDD: RDD[Point],
523     featuresDF: DataFrame,
524     ks: Seq[Int],
525     baseSeed: Long,
526     nRepeats: Int,
527     maxPoints: Option[Int]
528 ): Seq[ResultRow] = {
529
530     ks.flatMap { k =>
531
532         val m =
533             BucketManager.recommendedM(k)
534
535         (1 to nRepeats).flatMap { run =>
536
537             val seed =
538                 baseSeed + run - 1
539
540             println(
541                 s"[QualityMetrics] k-sweep: " +
542                 s"k=$k m=$m run=$run/$nRepeats seed=$seed"
543             )
544
545             val sequential =
546                 sequentialCostFromFile(
547                     path = path,
548                     k = k,
549                     m = m,
550                     seed = seed,
551                     maxPoints = maxPoints
552                 )
553
554             val distributed =
555                 distributedCost(
556                     pointsRDD,
557                     k,
558                     m,
559                     seed
560                 )
561
562             val mllib =
563                 mllibCost(
564                     featuresDF,
565                     k,
566                     seed
567                 )
568
569             Seq(
570
571                 ResultRow(
572                     datasetName,
573                     "Sequential StreamKM++",
574                     k,
575                     m,
576                     run,
577                     seed,
578                     sequential
579                 ),
580
581                 ResultRow(
582                     datasetName,
583                     "Spark StreamKM++",

```



```

584         k,
585         m,
586         run,
587         seed,
588         distributed
589     ),
590
591     ResultRow(
592         datasetName,
593         "Spark MLlib KMeans",
594         k,
595         m,
596         run,
597         seed,
598         mllib
599     )
600 )
601 }
602 }
603 }
604
605
606 // =====
607 // Experience B : effet de m
608 // =====
609
610 /**
611  * Fait varier la taille du coreset m avec k fixe.
612  */
613 def runMSweep(
614     path: String,
615     datasetName: String,
616     pointsRDD: RDD[Point],
617     k: Int,
618     ms: Seq[Int],
619     baseSeed: Long,
620     nRepeats: Int,
621     maxPoints: Option[Int]
622 ): Seq[ResultRow] = {
623
624     ms.flatMap { m =>
625
626         (1 to nRepeats).flatMap { run =>
627
628             val seed =
629                 baseSeed + run - 1
630
631             println(
632                 s"[QualityMetrics] m-sweep: " +
633                 s"k=$k m=$m run=$run/$nRepeats seed=$seed"
634             )
635
636             val sequential =
637                 sequentialCostFromFile(
638                     path = path,
639                     k = k,
640                     m = m,
641                     seed = seed,
642                     maxPoints = maxPoints
643                 )
644
645             val distributed =
646                 distributedCost(

```

```

647         pointsRDD,
648         k,
649         m,
650         seed
651     )
652
653     Seq(
654
655         ResultRow(
656             datasetName,
657             "Sequential StreamKM++",
658             k,
659             m,
660             run,
661             seed,
662             sequential
663         ),
664
665         ResultRow(
666             datasetName,
667             "Spark StreamKM++",
668             k,
669             m,
670             run,
671             seed,
672             distributed
673         )
674     )
675 }
676 }
677 }
678
679
680 // =====
681 // CSV
682 // =====
683
684 /**
685  * Echappe correctement les champs CSV contenant des virgules,
686  * des guillemets ou des retours a la ligne.
687  */
688 private def csvField(
689     value: String
690 ): String = {
691
692     if (
693         value.exists { c =>
694             c == ',' ||
695             c == '"' ||
696             c == '\n' ||
697             c == '\r'
698         }
699     ) {
700
701         "\"" +
702             value.replace("\"", "\\\"") +
703             "\""
704
705     } else {
706         value
707     }
708 }
709

```

```

710
711 /**
712  * Ecrit les resultats dans un fichier CSV unique.
713  */
714 def writeCsv(
715     path: String,
716     rows: Seq[ResultRow]
717 ): Unit = {
718
719     val file =
720         new File(path)
721
722     Option(
723         file.getParentFile
724     ).foreach(
725         _.mkdirs()
726     )
727
728     val writer =
729         new PrintWriter(
730             new OutputStreamWriter(
731                 new FileOutputStream(file),
732                 StandardCharsets.UTF_8
733             )
734         )
735
736     try {
737
738         writer.println(
739             "dataset,method,k,m,run,seed,cost"
740         )
741
742         rows.foreach { row =>
743
744             writer.println(
745                 Seq(
746                     csvField(row.dataset),
747                     csvField(row.method),
748                     row.k.toString,
749                     row.m.toString,
750                     row.run.toString,
751                     row.seed.toString,
752                     row.cost.toString
753                 ).mkString(",")
754             )
755         }
756
757     } finally {
758         writer.close()
759     }
760 }
761
762
763 // =====
764 // Arguments
765 // =====
766
767 /**
768  * Lit un entier positif depuis les arguments.
769  *
770  * Une valeur <= 0 est consideree comme absente.
771  */
772 private def positiveInt(

```

```

773     args: Array[String],
774     index: Int
775 ): Option[Int] = {
776
777     args
778     .lift(index)
779     .flatMap { value =>
780         Try(value.toInt).toOption
781     }
782     .filter(_ > 0)
783 }
784
785
786 // =====
787 // Main
788 // =====
789
790 /**
791  * Utilisation :
792  *
793  * Smoke test 20 000 points :
794  *
795  * sbt "runMain streamkm.experiments.QualityMetrics \
796  * data/covertime/covtype.data 20000 1"
797  *
798  *
799  * Dataset complet, une repetition :
800  *
801  * sbt "runMain streamkm.experiments.QualityMetrics \
802  * data/covertime/covtype.data 0 1"
803  *
804  *
805  * Arguments :
806  *
807  * args(0) : chemin du fichier
808  *
809  * args(1) : nombre maximal de points
810  *           0 = dataset complet
811  *
812  * args(2) : nombre de repetitions
813  */
814 def main(
815     args: Array[String]
816 ): Unit = {
817
818     val path =
819         args.headOption.getOrElse(
820             "data/covertime/covtype.data"
821         )
822
823     val maxPoints =
824         positiveInt(
825             args,
826             1
827         )
828
829     val nRepeats =
830         positiveInt(
831             args,
832             2
833         ).getOrElse(1)
834
835

```

```

836 val spark =
837     SparkSession
838         .builder()
839         .appName(
840             "StreamKM++ Quality Experiments"
841         )
842         .master("local[*]")
843         .config(
844             "spark.ui.enabled",
845             "false"
846         )
847         .config(
848             "spark.sql.shuffle.partitions",
849             DefaultPartitions.toString
850         )
851         .getOrCreate()
852
853
854 // Les logs Spark restent tres bavards.
855 // WARN suffit pour nos experiences locales.
856 spark.sparkContext.setLogLevel("WARN")
857
858
859 try {
860
861     println(
862         s"[QualityMetrics] Loading $path"
863     )
864
865     println(
866         s"[QualityMetrics] maxPoints=$maxPoints"
867     )
868
869     println(
870         s"[QualityMetrics] partitions=$DefaultPartitions"
871     )
872
873
874 // -----
875 // Chargement distribue direct du fichier
876 // -----
877
878 val pointsRDD =
879     loadCovertTypeRDD(
880         spark = spark,
881         path = path,
882         dim = Dim,
883         numPartitions = DefaultPartitions,
884         maxPoints = maxPoints
885     )
886     .persist(
887         StorageLevel.MEMORY_AND_DISK
888     )
889
890
891 try {
892
893     /*
894     * count() force le chargement du RDD.
895     *
896     * A partir de ce moment, les experiences reutilisent les donnees
897     * mises en cache plutot que de relire le fichier a chaque operation
898     * Spark.

```

```

898     */
899     val numberOfPoints =
900         pointsRDD.count()
901
902
903     require(
904         numberOfPoints > 0,
905         s"No valid point loaded from $path"
906     )
907
908
909     println(
910         s"[QualityMetrics] $numberOfPoints points loaded."
911     )
912
913     println(
914         s"[QualityMetrics] ${pointsRDD.getNumPartitions} Spark partitions."
915     )
916
917
918     // -----
919     // DataFrame MLlib construit une seule fois
920     // -----
921
922     val featuresDF =
923         toFeaturesDataFrame(
924             spark,
925             pointsRDD
926         )
927         .persist(
928             StorageLevel.MEMORY_AND_DISK
929         )
930
931
932     try {
933
934         /*
935          * Force le cache du DataFrame avant les differents appels MLlib.
936          */
937         featuresDF.count()
938
939
940         // =====
941         // Experience k-sweep
942         // =====
943
944         val kSweepRows =
945             runKSweep(
946                 spark = spark,
947                 path = path,
948                 datasetName = "Coverttype",
949                 pointsRDD = pointsRDD,
950                 featuresDF = featuresDF,
951                 ks = KValues,
952                 baseSeed = DefaultSeed,
953                 nRepeats = nRepeats,
954                 maxPoints = maxPoints
955             )
956
957
958         writeCsv(
959             "results/quality_k_sweep_coverttype.csv",
960             kSweepRows

```

```

961         )
962
963
964         // =====
965         // Experience m-sweep
966         // =====
967
968         val mSweepRows =
969             runMSweep(
970                 path = path,
971                 datasetName = "Coverttype",
972                 pointsRDD = pointsRDD,
973                 k = 25,
974                 ms = MValues,
975                 baseSeed = DefaultSeed,
976                 nRepeats = nRepeats,
977                 maxPoints = maxPoints
978             )
979
980
981         writeCsv(
982             "results/quality_m_sweep_coverttype.csv",
983             mSweepRows
984         )
985
986
987         println(
988             "[QualityMetrics] Done."
989         )
990
991         println(
992             "  -> results/quality_k_sweep_coverttype.csv"
993         )
994
995         println(
996             "  -> results/quality_m_sweep_coverttype.csv"
997         )
998
999
1000     } finally {
1001
1002         // Libere le cache du DataFrame MLlib.
1003         featuresDF.unpersist()
1004     }
1005
1006
1007     } finally {
1008
1009         // Libere le RDD Coverttype du cache Spark.
1010         pointsRDD.unpersist()
1011     }
1012
1013
1014     } finally {
1015
1016         spark.stop()
1017     }
1018 }
1019 }

```

Listing A.8 – streamkm/experiments/QualityMetrics.scala

A.0.6 Fonction Main

Le main se lance avec :

```
1 sbt run      # choisir streamkm.demo.Main si plusieurs mains sont proposes

1 package streamkm.demo
2
3 import scala.util.Random
4
5 import streamkm.core.{Distance, KMeans, Point}
6 import streamkm.coreset.BucketManager
7
8 /**
9  * Démonstrateur de bout en bout pour les couches E1 ('streamkm.core') et
10 * E2 ('streamkm.coreset'), en l'absence de la couche 'streamkm.spark' (E4,
11 * volontairement hors scope de cette session).
12 *
13 * Rejoue en un seul thread l'algorithme StreamKM++ complet tel que décrit au
14 * §2.3.4 de la thèse (« The StreamKM++ Algorithm ») :
15 * 1. consommer le flux point par point via le merge-and-reduce (algo 2.5) ;
16 * 2. extraire le coreset final (union des buckets non vides, §2.3.4) ;
17 * 3. appliquer k-means++ (seeding  $D^2$  algo 2.2 + Lloyd algo 2.1) sur ce
18 *    coreset et garder le meilleur des 'nRestarts' redémarrages - c'est la
19 *    même politique que l'opérateur 'FlatMapToKmeansPP' du §4.1 de la thèse.
20 *
21 * C'est exactement le scénario « parallélisme 1 » qui sert de référence à
22 * l'expérience E7 du METHODO (fusion séquentielle vs 'treeAggregate').
23 */
24 object Main {
25
26   /**
27    * Génère un mélange de 'k' gaussiennes isotropes en dimension 'd', à vérité
28    * terrain connue. Les vrais centres sont placés sur une grille de pas
29    * 'spread' pour garantir leur séparabilité - reproduit ici la même
30    * construction que 'TestData.gaussianMixture' (scope test), qu'on ne peut
31    * pas référencer depuis 'src/main'.
32    */
33   private def gaussianMixture(
34     n: Int, k: Int, d: Int, sigma: Double, spread: Double, rng: Random
35   ): (Array[Point], Array[Array[Double]]) = {
36     val trueCenters = Array.tabulate(k) { c =>
37       Array.tabulate(d)(j => ((c + 1) * (j + 1) % k).toDouble * spread)
38     }
39     val points = Array.tabulate(n) { _ =>
40       val ctr = trueCenters(rng.nextInt(k))
41       Point(Array.tabulate(d)(j => ctr(j) + rng.nextGaussian() * sigma))
42     }
43     (points, trueCenters)
44   }
45
46   def main(args: Array[String]): Unit = {
47     val n      = 200000 // taille du flux
48     val k      = 5
49     val d      = 2
50     val m      = BucketManager.recommendedM(k) // 200·k, thèse §2.3.4
51               clarification 1
52     val seed   = 42L
53     val rng    = new Random(seed)
54
55     val (points, trueCenters) = gaussianMixture(n, k, d, sigma = 0.5, spread =
56       20.0, rng)
57
58     println(s"StreamKM++ (démon E1/E2, sans Spark) - n=$n, k=$k, d=$d, m=$m")
59   }
60 }
```



```

57 // --- Étape 1 : consommation du flux, un point à la fois (algo 2.5) ----
58 // Un seul BucketManager = un seul subtask Flink / une seule partition Spark
59
60 // C'est la structure que E4 fera tourner en parallèle via treeAggregate.
61 val t0 = System.nanoTime()
62 val manager = BucketManager(m, seed)
63 manager.insertAll(points.iterator)
64 val tIngest = (System.nanoTime() - t0) / 1e6
65
66 // --- Étape 2 : extraction du coreset final (union des buckets, §2.3.4) -
67 val t1 = System.nanoTime()
68 val coreset = manager.extractCoreset()
69 val tExtract = (System.nanoTime() - t1) / 1e6
70
71 // --- Étape 3 : k-means++ sur le coreset (5 redémarrages, §4.1) -----
72 val t2 = System.nanoTime()
73 val model = KMeans.fit(coreset, k, nRestarts = 5, seed = seed)
74 val tKmeans = (System.nanoTime() - t2) / 1e6
75
76 // Coût mesuré sur les DONNÉES COMPLÈTES avec les centres issus du coreset :
77 // seule comparaison honnête (thèse §4.2,  $cost(P, C) = \sum D^2(x, C)$ ), le coût
78 // évalué sur le coreset lui-même n'est pas comparable à un coût batch.
79 val costOnFullData = KMeans.cost(points, model.centers)
80 val batchCost = KMeans.fit(points, k, nRestarts = 5, seed = seed).cost
81 val maxDeviation = trueCenters.map(tc => math.sqrt(Distance.nearest(tc,
82     model.centers)._2)).max
83
84 println(f"Points vus par le manager : ${manager.numPointsSeen}%.0f (attendu
85     $n)")
86 println(f"Coreset final : ${coreset.length} points (m=$m)")
87 println(f"Temps ingestion flux : $tIngest%.1f ms")
88 println(f"Temps extraction coreset : $tExtract%.1f ms")
89 println(f"Temps k-means++ (coreset) : $tKmeans%.1f ms")
90 println()
91 println(f"Coût (coreset) sur données complètes : $costOnFullData%.2f")
92 println(f"Coût k-means++ batch (référence) : $batchCost%.2f (ratio ${
93     costOnFullData / batchCost}%.3f)")
94 println(f"Écart max centre trouvé / vrai centre : $maxDeviation%.3f")
95 println()
96 println("Centres trouvés :")
97 model.centers.foreach(c => println(" " + c.map(v => f"$v%.2f").mkString("("
98     , " ", " ", " ")))
99 }
100 }

```

Listing A.9 – streamkm/demo/Main.scala : démonstrateur section 1/section 2 sur données synthétiques

A.0.7 Suite de tests

```

1 package streamkm
2
3 import scala.util.Random
4
5 import streamkm.core.{Distance, Point}
6
7 /**
8  * Générateur de données synthétiques à vérité terrain.
9  *
10  * C'est l'outil de l'expérience E1 (correction) : on connaît les vrais centres,
    donc on

```

```

11  * peut vérifier que l'algorithme les retrouve, ce qu'aucun jeu réel ne permet.
    Il servira
12  * aussi en E3/E4 pour faire varier la taille des données à volonté sans dé
    pendre d'un
13  * téléchargement.
14  */
15  object TestData {
16
17      final case class Mixture(points: Array[Point], trueCenters: Array[Array[Double]
18          ])
19
20      /**
21       * Mélange de 'k' gaussiennes isotropes de dimension 'd', dont les centres
22       * sont placés
23       * sur une grille de pas 'spread' pour garantir la séparabilité.
24       */
25      def gaussianMixture(
26          n:      Int,
27          k:      Int,
28          d:      Int,
29          sigma:  Double = 0.5,
30          spread: Double = 20.0,
31          seed:   Long   = 12345L
32      ): Mixture = {
33          val rng = new Random(seed)
34
35          val centers = Array.tabulate(k) { c =>
36              Array.tabulate(d)(j => ((c + 1) * (j + 1) % k).toDouble * spread)
37          }
38
39          val pts = Array.tabulate(n) { _ =>
40              val c = rng.nextInt(k)
41              val ctr = centers(c)
42              Point(Array.tabulate(d)(j => ctr(j) + rng.nextGaussian() * sigma))
43          }
44
45          Mixture(pts, centers)
46      }
47
48      /** Somme des poids d'un ensemble de points. */
49      def totalMass(pts: Array[Point]): Double = {
50          var s = 0.0
51          var i = 0
52          while (i < pts.length) { s += pts(i).weight; i += 1 }
53          s
54      }
55
56      /**
57       * Distance maximale entre chaque vrai centre et le centre estimé le plus
58       * proche.
59       * Sert à vérifier qu'un clustering a bien « retrouvé » la structure sous-
60       * jacente,
61       * indépendamment de l'ordre des centres.
62       */
63      def maxCenterDeviation(
64          trueCenters: Array[Array[Double]],
65          found:      Array[Array[Double]]
66      ): Double = {
67          var worst = 0.0
68          var i = 0
69          while (i < trueCenters.length) {
70              val d = math.sqrt(Distance.nearest(trueCenters(i), found)._2)
71              if (d > worst) worst = d
72          }
73      }

```

```

68     i += 1
69   }
70   worst
71 }
72 }

```

Listing A.10 – TestData.scala : générateur de données synthétiques à vérité terrain

```

1  package streamkm.core
2
3  import scala.util.Random
4
5  import org.scalatest.funsuite.AnyFunSuite
6  import org.scalatest.matchers.should.Matchers
7
8  import streamkm.TestData
9
10 /**
11  * Critères de sortie de l'étape E1.
12  * Tant qu'un de ces tests est rouge, on ne commence pas E2.
13  */
14 class KMeansSpec extends AnyFunSuite with Matchers {
15
16   // -----
17   // Test 1 - valeur analytique connue
18   // -----
19   test("cost : trois points colinéaires, k=1, valeur analytique") {
20     val pts = Array(Point.of(-1.0), Point.of(0.0), Point.of(1.0))
21     // Centre optimal = moyenne = 0 ; SSE = 1 + 0 + 1 = 2
22     KMeans.cost(pts, Array(Array(0.0))) shouldBe 2.0 +- 1e-12
23
24     val model = KMeans.fit(pts, k = 1, nRestarts = 3, seed = 1L)
25     model.centers(0)(0) shouldBe 0.0 +- 1e-9
26     model.cost shouldBe 2.0 +- 1e-9
27   }
28
29   // -----
30   // Test 2 - deux gaussiennes bien séparées
31   // -----
32   test("lloyd : retrouve deux gaussiennes séparées") {
33     val rng = new Random(7L)
34     val pts = Array.tabulate(2000) { i =>
35       val c = if (i % 2 == 0) 0.0 else 10.0
36       Point(Array(c + rng.nextGaussian() * 0.1, c + rng.nextGaussian() * 0.1))
37     }
38
39     val model = KMeans.fit(pts, k = 2, nRestarts = 5, seed = 3L)
40     val dev = TestData.maxCenterDeviation(Array(Array(0.0, 0.0), Array(10.0,
41       10.0)), model.centers)
42     dev should be < 0.05
43   }
44
45   // -----
46   // Test 3 - monotonie du coût
47   // -----
48   test("lloyd : le coût ne remonte jamais") {
49     // L'assertion de monotonie est dans la boucle de 'lloyd' et n'est active
50     // qu'avec -ea (cf. build.sbt). Ce test la déclenche sur une initialisation
51     // volontairement mauvaise, et vérifie en plus la propriété de l'extérieur.
52     val mix = TestData.gaussianMixture(n = 3000, k = 4, d = 3, seed = 99L)
53     val badInit = Array.fill(4)(Array.fill(3)(0.0))
54     val before = KMeans.cost(mix.points, badInit)
55     val model = KMeans.lloyd(mix.points, badInit, maxIter = 50)
56     model.cost should be <= before

```

```

56 }
57
58 // -----
59 // Test 4 - pondération (LE test qui attrape le bug classique)
60 // -----
61 test("lloyd : un point de poids w équivaut à w copies du même point") {
62     val a = Array(0.0, 0.0)
63     val b = Array(10.0, 0.0)
64
65     // Version « répliquée » : 3 copies de a, 1 de b
66     val replicated = Array(Point(a), Point(a), Point(a), Point(b))
67     // Version « pondérée » : a de poids 3, b de poids 1
68     val weighted = Array(Point(a, 3.0), Point(b, 1.0))
69
70     val init = Array(Array(-5.0, 0.0))
71
72     val mR = KMeans.lloyd(replicated, init, maxIter = 50)
73     val mW = KMeans.lloyd(weighted, init, maxIter = 50)
74
75     // Barycentre attendu :  $(3 \cdot 0 + 1 \cdot 10) / 4 = 2.5$ 
76     mR.centers(0)(0) shouldBe 2.5 +- 1e-9
77     mW.centers(0)(0) shouldBe 2.5 +- 1e-9
78     mW.cost shouldBe mR.cost +- 1e-9
79 }
80
81 // -----
82 // Test 5 - seeding
83 // -----
84 test("seedPlusPlus : produit k centres, bornés par n, de la bonne dimension")
85 {
86     val mix = TestData.gaussianMixture(n = 500, k = 5, d = 4, seed = 21L)
87
88     val s = KMeans.seedPlusPlus(mix.points, k = 5, new Random(1L))
89     s.length shouldBe 5
90     s.foreach(_.length shouldBe 4)
91
92     // k > n : on ne peut pas produire plus de centres que de points
93     val tiny = Array(Point.of(0.0), Point.of(1.0))
94     KMeans.seedPlusPlus(tiny, k = 10, new Random(1L)).length shouldBe 2
95 }
96
97 test("seedPlusPlus : le seeding D2 sépare les groupes éloignés") {
98     // 3 amas très distants : sur 20 tirages, le seeding doit quasi toujours
99     // placer un centre dans chaque amas - c'est toute la valeur de k-means++.
100     val rng = new Random(5L)
101     val pts = Array.tabulate(900) { i =>
102         val c = (i % 3) * 1000.0
103         Point(Array(c + rng.nextGaussian()))
104     }
105
106     var successes = 0
107     (0 until 20).foreach { t =>
108         val s = KMeans.seedPlusPlus(pts, 3, new Random(t.toLong))
109         val groups = s.map(c => math.round(c(0) / 1000.0)).distinct
110         if (groups.length == 3) successes += 1
111     }
112     successes should be >= 18
113 }
114
115 // -----
116 // Test 6 - fit garde bien le meilleur redémarrage
117 // -----
118 test("fit : le coût décroît (au sens large) avec le nombre de redémarrages") {

```

```

118     val mix  = TestData.gaussianMixture(n = 2000, k = 6, d = 2, sigma = 2.0,
119         seed = 33L)
119     val one  = KMeans.fit(mix.points, k = 6, nRestarts = 1, seed = 8L)
120     val five = KMeans.fit(mix.points, k = 6, nRestarts = 5, seed = 8L)
121     // Même graine : les 5 redémarrages incluent celui de nRestarts=1.
122     five.cost should be <= one.cost + 1e-9
123 }
124 }

```

Listing A.11 – core/KMeansSpec.scala

```

1  package streamkm.coreset
2
3  import scala.util.Random
4
5  import org.scalatest.funsuite.AnyFunSuite
6  import org.scalatest.matchers.should.Matchers
7
8  import streamkm.TestData
9  import streamkm.core.{KMeans, Point}
10
11  /**
12   * Critères de sortie de l'étape E2.
13   *
14   * Le test central est la CONSERVATION DE LA MASSE. Un coreset dont la somme des
15   * poids
16   * ne vaut pas n est faux, quelle que soit son apparence - et c'est le seul
17   * symptôme
18   * détectable à coup sûr, parce qu'un coreset légèrement erroné produit tout de
19   * même
20   * des centres plausibles.
21   */
22  class CoresetSpec extends AnyFunSuite with Matchers {
23
24     // =====
25     // CoresetTree
26     // =====
27
28     test("CoresetTree : renvoie l'ensemble tel quel si n <= m") {
29         val pts = TestData.gaussianMixture(n = 50, k = 3, d = 2, seed = 1L).points
30         val cs  = CoresetTree.build(pts, m = 100, new Random(1L))
31         cs.length shouldBe 50
32         TestData.totalMass(cs) shouldBe 50.0 +- 1e-9
33     }
34
35     test("CoresetTree : produit au plus m points") {
36         val pts = TestData.gaussianMixture(n = 5000, k = 5, d = 3, seed = 2L).points
37         for (m <- Seq(10, 100, 1000)) {
38             CoresetTree.build(pts, m, new Random(m.toLong)).length should be <= m
39         }
40     }
41
42     test("CoresetTree : conserve exactement la masse totale") {
43         val pts = TestData.gaussianMixture(n = 5000, k = 5, d = 3, seed = 3L).points
44         for (m <- Seq(10, 100, 500)) {
45             val cs = CoresetTree.build(pts, m, new Random(m.toLong))
46             withClue(s"m=$m : ") {
47                 TestData.totalMass(cs) shouldBe 5000.0 +- 1e-6
48             }
49         }
50     }
51
52     test("CoresetTree : conserve la masse sur des points déjà pondérés") {
53         val rng = new Random(4L)

```

```

51     val pts = Array.tabulate(3000)(_ =>
52         Point(Array(rng.nextGaussian() * 10, rng.nextGaussian() * 10), 1.0 + rng.
           nextInt(9))
53     )
54     val expected = TestData.totalMass(pts)
55     val cs       = CoresetTree.build(pts, m = 200, new Random(5L))
56     TestData.totalMass(cs) shouldBe expected +- 1e-6
57 }
58
59 test("CoresetTree : gère un ensemble de points tous identiques") {
60     // Coût global nul : la boucle de construction doit s'arrêter sans boucler à
        // l'infini.
61     val pts = Array.fill(1000)(Point(Array(3.0, 4.0)))
62     val cs  = CoresetTree.build(pts, m = 50, new Random(6L))
63     cs.length should be <= 50
64     TestData.totalMass(cs) shouldBe 1000.0 +- 1e-9
65     cs.foreach { p => p.coords(0) shouldBe 3.0; p.coords(1) shouldBe 4.0 }
66 }
67
68 test("CoresetTree : le coreset préserve la qualité du clustering") {
69     val mix  = TestData.gaussianMixture(n = 20000, k = 5, d = 2, sigma = 0.5,
        seed = 7L)
70     val full = KMeans.fit(mix.points, k = 5, nRestarts = 5, seed = 11L)
71
72     val cs      = CoresetTree.build(mix.points, m = 200, new Random(12L))
73     val onCoreset = KMeans.fit(cs, k = 5, nRestarts = 5, seed = 11L)
74     // On évalue les centres issus du coreset sur les DONNÉES COMPLÈTES : c'est
        // la seule
75     // comparaison honnête (le coût mesuré sur le coreset n'est pas comparable).
76     val costViaCores = KMeans.cost(mix.points, onCoreset.centers)
77
78     withClue(s"complet=${full.cost}, via coreset=$costViaCores : ") {
79         costViaCores should be <= full.cost * 1.10
80     }
81 }
82
83 // =====
84 // BucketManager
85 // =====
86
87 test("BucketManager : compte correctement les points consommés") {
88     val bm = BucketManager(m = 100, seed = 1L)
89     val pts = TestData.gaussianMixture(n = 5000, k = 4, d = 2, seed = 8L).points
90     bm.insertAll(pts)
91     bm.numPointsSeen shouldBe 5000.0 +- 1e-9
92 }
93
94 test("BucketManager : respecte les invariants de taille des buckets") {
95     val bm = BucketManager(m = 64, seed = 2L)
96     val pts = TestData.gaussianMixture(n = 4000, k = 4, d = 2, seed = 9L).points
97     // On vérifie après chaque insertion, pas seulement à la fin.
98     pts.foreach { p =>
99         bm.insert(p)
100         bm.checkInvariants() shouldBe true
101     }
102 }
103
104 test("BucketManager : le nombre de niveaux croît logarithmiquement") {
105     val m = 100
106     val bm = BucketManager(m, seed = 3L)
107     bm.insertAll(TestData.gaussianMixture(n = 100000, k = 3, d = 2, seed = 10L).
        points)
108     //  $O(\log_2(n/m)) + 2 \approx 10 + 2$ . On laisse une marge confortable ; le point du

```

```

    test est
109 // qu'on ne soit PAS en  $O(n/m) = 1000$ .
110 bm.numLevels should be <= 16
111 }
112
113 test("BucketManager : union() et extractCoreset() conservent la masse") {
114     val bm = BucketManager(m = 128, seed = 4L)
115     bm.insertAll(TestData.gaussianMixture(n = 10000, k = 5, d = 3, seed = 11L).
        points)
116
117     TestData.totalMass(bm.union())           shouldBe 10000.0 +- 1e-6
118     TestData.totalMass(bm.extractCoreset())  shouldBe 10000.0 +- 1e-6
119     bm.extractCoreset().length               should be <= 128
120 }
121
122 test("BucketManager : extractCoreset ne détruit pas l'état") {
123     val bm = BucketManager(m = 64, seed = 5L)
124     bm.insertAll(TestData.gaussianMixture(n = 5000, k = 3, d = 2, seed = 12L).
        points)
125
126     val first = bm.extractCoreset()
127     bm.insertAll(TestData.gaussianMixture(n = 5000, k = 3, d = 2, seed = 13L).
        points)
128     val second = bm.extractCoreset()
129
130     first.length should be <= 64
131     bm.numPointsSeen shouldBe 10000.0 +- 1e-9
132     TestData.totalMass(second) shouldBe 10000.0 +- 1e-6
133 }
134
135 test("BucketManager : merge conserve la masse des deux opérandes") {
136     val a = BucketManager(m = 100, seed = 6L)
137     val b = BucketManager(m = 100, seed = 7L)
138     a.insertAll(TestData.gaussianMixture(n = 4000, k = 4, d = 2, seed = 14L).
        points)
139     b.insertAll(TestData.gaussianMixture(n = 6000, k = 4, d = 2, seed = 15L).
        points)
140
141     val merged = a.merge(b)
142     merged.numPointsSeen shouldBe 10000.0 +- 1e-9
143     TestData.totalMass(merged.union()) shouldBe 10000.0 +- 1e-6
144
145     // Les opérandes ne sont pas modifiées (indispensable pour un combOp de
        treeAggregate,
146     // que Spark peut ré-exécuter en cas de perte de tâche).
147     a.numPointsSeen shouldBe 4000.0 +- 1e-9
148     b.numPointsSeen shouldBe 6000.0 +- 1e-9
149 }
150
151 test("BucketManager : insertion refusée après merge") {
152     val a = BucketManager(m = 32, seed = 8L)
153     val b = BucketManager(m = 32, seed = 9L)
154     a.insertAll(TestData.gaussianMixture(n = 500, k = 2, d = 2, seed = 16L).
        points)
155     b.insertAll(TestData.gaussianMixture(n = 500, k = 2, d = 2, seed = 17L).
        points)
156
157     an[IllegalStateException] should be thrownBy a.merge(b).insert(Point.of(0.0,
        0.0))
158 }
159
160 // =====
161 // Bout en bout : StreamKM++ séquentiel

```

```

162 // =====
163
164 test("StreamKM++ séquentiel : qualité comparable au k-means++ batch") {
165   // C'est l'algorithme complet du §2.3.4, en un seul thread : merge-and-
166     reduce sur
167   // tout le flux, puis k-means++ sur l'union finale des buckets.
168   // Il servira de référence (« parallélisme 1 ») pour toutes les expériences
169     de E7.
170   val k = 5
171   val m = BucketManager.recommendedM(k) // 200·k, cf. §2.3.4
172   val mix = TestData.gaussianMixture(n = 50000, k = k, d = 2, sigma = 0.5,
173     seed = 18L)
174
175   val bm = BucketManager(m, seed = 19L)
176   bm.insertAll(mix.points)
177
178   val finalCoreset = bm.union()
179   val streamModel = KMeans.fit(finalCoreset, k, nRestarts = 5, seed = 20L)
180   val streamCost = KMeans.cost(mix.points, streamModel.centers)
181
182   val batchCost = KMeans.fit(mix.points, k, nRestarts = 5, seed = 20L).cost
183
184   withClue(s"batch=$batchCost, stream=$streamCost : ") {
185     streamCost should be <= batchCost * 1.10
186   }
187   TestData.maxCenterDeviation(mix.trueCenters, streamModel.centers) should be
188     < 0.5
189 }

```

Listing A.12 – coreset/CoresetSpec.scala

```

1 package streamkm.spark
2
3 import org.apache.spark.sql.{SQLContext, SparkSession}
4 import org.apache.spark.sql.execution.streaming.MemoryStream
5 import org.scalatest.BeforeAndAfterAll
6 import org.scalatest.funSuite.AnyFunSuite
7 import org.scalatest.matchers.should.Matchers
8
9 import streamkm.TestData
10 import streamkm.core.{KMeans, KMeansModel}
11 import streamkm.coreset.BucketManager
12
13 /**
14  * Modèle de composition d'erreur pour 'treeReduce' (thèse §2.3.4, observation 2
15  * :
16  * « un  $\varepsilon$ -coreset d'un  $\delta$ -coreset est un  $(\varepsilon+\delta+\varepsilon\delta)$ -coreset », c.-à-d.  $(1+\varepsilon)(1+\delta)-1$ 
17  * -
18  * l'erreur se COMPOSE multiplicativement, pas additivement).
19  *
20  * IMPORTANT - limite honnête : la thèse ne donne aucune formule fermée pour l' $\varepsilon$ 
21  * d'un
22  * coreset produit par 'CoresetTree' (§2.3.4, point 3 : « the authors do not
23  * prove that
24  * the coreset tree structure is indeed a  $(k, \varepsilon)$ -coreset »). Le seul  $\varepsilon$  chiffré de
25  * la thèse
26  * concerne 'AdaptiveCoreset' (théorème 2.1), avec une constante  $O(d)$  non précis
27  * ée - donc
28  * inutilisable numériquement. Ce qu'on PEUT dériver rigoureusement, c'est la
29  * loi de
30  * composition (multiplicative) et le nombre de niveaux de composition qu'
31  * introduit

```



```

24 * 'treeReduce'. Pour la magnitude d'un niveau isolé, on réutilise la seule
    valeur dont on
25 * dispose : l' $\epsilon$  empirique déjà validé par 'CoresetSpec' ("le coreset préserve
    la qualité
26 * du clustering", 1 seul niveau de réduction, tolérance choisie à 10 % dans ce
    repo).
27 */
28 private[spark] object CoresetErrorModel {
29
30     /**
31     * Facteur de regroupement utilisé par 'RDD.treeReduce'/'treeAggregate' en
        interne
32     * (implémentation Spark : 'scale = max(2, ceil(numPartitions^(1/depth)))').
33     */
34     def scaleFactor(numPartitions: Int, depth: Int): Int =
35         math.max(2, math.ceil(math.pow(numPartitions.toDouble, 1.0 / depth)).toInt)
36
37     /**
38     * Nombre MAXIMUM de fusions 'BucketManager.merge' séquentielles que peut
        subir la
39     * contribution d'un point donné, dans le pire cas, à travers un 'treeReduce'
        de
40     * profondeur 'depth' sur 'numPartitions' partitions.
41     *
42     * Dérivation : à chaque palier (il y en a au plus 'depth'), Spark regroupe
        environ
43     * 'scale' éléments par clé via 'reduceByKey', ce qui replie ce groupe par une
        CHAÎNE de
44     * 'scale - 1' applications séquentielles de 'combOp' (chaque application
        composant à
45     * nouveau l'erreur, cf. observation 2 ci-dessus). D'où la borne 'depth * (
        scale - 1)'.
46     *
47     * Ce n'est PAS une garantie publiée par Spark (treeReduce ne documente pas de
        borne de
48     * composition d'erreur, pour cause : ses 'U' ne sont pas des coreset en géné
        ral) - c'est
49     * une estimation dérivée de sa stratégie de regroupement interne, pour un
        pire cas.
50     */
51     def worstCaseCompositionLevels(numPartitions: Int, depth: Int): Int = {
52         val scale = scaleFactor(numPartitions, depth)
53         depth * (scale - 1)
54     }
55
56     /** (1+ $\epsilon$ )levels - 1 : composition multiplicative d'une erreur  $\epsilon$  sur 'levels'
        niveaux. */
57     def compoundedTolerance(epsPerLevel: Double, levels: Int): Double =
58         math.pow(1.0 + epsPerLevel, levels) - 1.0
59
60     /**
61     * Tolérance relative attendue pour comparer le coût d'un 'mergeCoresets'
        distribué
62     * (sur 'numPartitions' partitions, profondeur 'depth') à une référence sé
        quentielle à un
63     * seul niveau de réduction.
64     *
65     * 'epsPerLevel = 0.10' : repris tel quel de 'CoresetSpec' (E2), qui valide dé
        jà qu'UNE
66     * réduction CoresetTree isolée dégrade le coût de moins de 10 % en pratique
        sur des
67     * mélanges gaussiens comparables. On ne réinvente pas cette calibration ici,
        on la

```

```

68     * compose.
69     */
70     def treeReduceTolerance(numPartitions: Int, depth: Int, epsPerLevel: Double =
71         0.10): Double =
72         compoundedTolerance(epsPerLevel, worstCaseCompositionLevels(numPartitions,
73             depth))
74 }
75 /**
76  * Critères de sortie de l'étape E4.
77  *
78  * Le test central (« non-régression distribué vs séquentiel ») est plus
79  * important ici que la conservation stricte de la masse (déjà couverte par
80  * E2) : il vérifie que distribuer StreamKM++ sur Spark ne dégrade pas la
81  * qualité du clustering par rapport à l'exécution séquentielle de référence
82  * ('demo.Main', ou le test "StreamKM++ séquentiel" de 'CoresetSpec').
83  *
84  * Les tests "état streaming" (fin de fichier) couvrent le risque le plus récent
85  * de la
86  * couche Spark : le double-comptage ou la perte de masse entre micro-batches
87  * successifs
88  * dans l'état 'global' maintenu par 'StreamKMPlusPlus.run'.
89  */
90 class StreamKMSpec extends AnyFunSuite with Matchers with BeforeAndAfterAll {
91     @transient private var spark: SparkSession = _
92
93     override def beforeAll(): Unit = {
94         spark = SparkSession.builder()
95             .appName("StreamKMSpec")
96             .master("local[4]")
97             .config("spark.ui.enabled", "false")
98             .config("spark.sql.shuffle.partitions", "4")
99             .getOrCreate()
100     }
101
102     override def afterAll(): Unit = {
103         if (spark != null) spark.stop()
104     }
105
106     // =====
107     // partialCoresets
108     // =====
109
110     test("partialCoresets : un coreset par partition, masse totale conservée") {
111         val mix = TestData.gaussianMixture(n = 8000, k = 4, d = 2, seed = 31L)
112         val rdd = spark.sparkContext.parallelize(mix.points, numSlices = 4)
113
114         val params = StreamKMParams(k = 4, m = 200)
115         val partials = StreamKMPlusPlus.partialCoresets(rdd, params).collect()
116
117         partials.length shouldBe 4
118         partials.foreach(_.length should be <= 200)
119
120         val totalMass = partials.iterator.flatten.map(_.weight).sum
121         totalMass shouldBe 8000.0 +- 1e-6
122     }
123
124     // =====
125     // mergeCoresets
126     // =====
127
128     test("mergeCoresets : conserve la masse totale, quel que soit le nombre de

```

```

    partitions") {
127   val mix      = TestData.gaussianMixture(n = 10000, k = 5, d = 2, seed = 32L)
128   val params = StreamKMParams(k = 5, m = BucketManager.recommendedM(5))
129
130   for (nParts <- Seq(1, 2, 8)) {
131     val rdd      = spark.sparkContext.parallelize(mix.points, numSlices =
132       nParts)
133     val coreset = StreamKMPlusPlus.mergeCoresets(rdd, params)
134     withClue(s"nParts=$nParts : ") {
135       coreset.map(_.weight).sum shouldBe 10000.0 +- 1e-6
136       coreset.length should be <= params.m
137     }
138   }
139 }
140
141 // =====
142 // LE TEST LE PLUS IMPORTANT : non-régression distribué vs séquentiel
143 // =====
144
145 test("mergeCoresets sur 8 partitions produit un clustering de qualité
146   comparable au séquentiel") {
147   val k      = 5
148   val m      = BucketManager.recommendedM(k)
149   val mix    = TestData.gaussianMixture(n = 40000, k = k, d = 2, sigma =
150     0.5, seed = 33L)
151   val numParts = 8
152   val treeDepth = 2 // valeur par défaut de mergeCoresets, non surchargée ici
153
154   // Référence séquentielle : un seul BucketManager, tout le flux d'un coup
155   // (même logique que le test "StreamKM++ séquentiel" de CoresetSpec, E2,
156   // et que demo.Main).
157   val seqBm = new BucketManager(m, seed = 40L)
158   seqBm.insertAll(mix.points)
159   val seqModel = KMeans.fit(seqBm.extractCoreset(), k, nRestarts = 5, seed =
160     41L)
161   val seqCost = KMeans.cost(mix.points, seqModel.centers)
162
163   // Version distribuée : mêmes données, réparties sur 'numParts' partitions
164   // Spark.
165   val rdd      = spark.sparkContext.parallelize(mix.points, numSlices =
166     numParts)
167   val distCoreset = StreamKMPlusPlus.mergeCoresets(rdd, StreamKMParams(k = k,
168     m = m, seed = 40L), depth = treeDepth)
169   val distModel = KMeans.fit(distCoreset, k, nRestarts = 5, seed = 41L)
170   val distCost = KMeans.cost(mix.points, distModel.centers)
171
172   // Tolérance DÉRIVÉE (pas codée en dur) de la loi de composition
173   // multiplicative des
174   // coresets (thèse §2.3.4, observation 2) appliquée au nombre de niveaux de
175   // fusion
176   // que 'treeReduce' introduit réellement pour numParts=8, depth=2 (
177   // CoresetErrorModel).
178   // Voir CLAUDE.md, décision 11, pour le calcul détaillé et la comparaison à
179   // l'ancienne valeur codée en dur (25 %).
180   val tolerance = CoresetErrorModel.treeReduceTolerance(numParts, treeDepth)
181
182   withClue(
183     s"séquentiel=$seqCost, distribué=$distCost, tolérance théorique=${
184       tolerance * 100}% : "
185   ) {
186     distCost should be <= seqCost * (1.0 + tolerance)
187   }
188   TestData.maxCenterDeviation(mix.trueCenters, distModel.centers) should be <

```

```

178     2.0
179 }
180 test("mergeCoresets : le coût ne se dégrade pas franchement entre 1 et 8
181     partitions") {
182     val k = 4
183     val m = BucketManager.recommendedM(k)
184     val mix = TestData.gaussianMixture(n = 20000, k = k, d = 2, sigma = 0.5,
185         seed = 34L)
186
187     val params = StreamKMParams(k = k, m = m, seed = 50L)
188
189     def costWith(nParts: Int): Double = {
190         val rdd = spark.sparkContext.parallelize(mix.points, numSlices =
191             nParts)
192         val coreset = StreamKMPlusPlus.mergeCoresets(rdd, params)
193         KMeans.cost(mix.points, KMeans.fit(coreset, k, nRestarts = 5, seed = 51L).
194             centers)
195     }
196
197     val cost1 = costWith(1)
198     val cost8 = costWith(8)
199
200     withClue(s"1 partition=$cost1, 8 partitions=$cost8 : ") {
201         // La thèse observe (§5.3.1, fig. 5.9) que le coût varie peu avec le
202         // parallélisme, et peut même s'améliorer légèrement (plus de listes de
203         // buckets = coresets plus fins). On vérifie ici l'absence de
204         // dégradation franche - même observation que E3bis du METHODODO, en
205         // version test de non-régression plutôt qu'expérience chiffrée.
206         math.abs(cost1 - cost8) should be <= 0.3 * math.max(cost1, cost8)
207     }
208 }
209
210 test("partialCoresets et mergeCoresets utilisent des graines distinctes par
211     partition") {
212     // Les deux fonctions dérivent la graine de chaque partition comme
213     // 'p.seed + idx' (cf. commentaire de StreamKMPlusPlus.partialCoresets).
214     // On vérifie ici que deux partitions contenant les MÊMES points ne
215     // produisent pas des coresets identiques point à point (ce qui trahirait
216     // une graine partagée) : c'est une garantie de non-corrélation, pas une
217     // propriété algorithmique de la thèse, mais elle conditionne la validité
218     // de l'expérience E3bis (isoler l'effet du nombre de partitions de tout
219     // effet de graine).
220     val pts = TestData.gaussianMixture(n = 2000, k = 3, d = 2, seed = 60L).
221         points
222     val rdd = spark.sparkContext.parallelize(pts ++ pts, numSlices = 2) // 2
223         copies identiques
224     val params = StreamKMParams(k = 3, m = 50)
225
226     val partials = StreamKMPlusPlus.partialCoresets(rdd, params).collect()
227     partials.length should be 2
228     // Si les deux partitions recevaient exactement les mêmes points originaux
229     // avec la même graine, leurs coresets seraient identiques coordonnée à
230     // coordonnée ; avec des graines distinctes, ce n'est presque sûrement pas
231     // le cas dès qu'un tirage aléatoire intervient (m=50 << n/2=2000, donc le
232     // coreset tree tire bien des échantillons aléatoires).
233     partials(0).map(_.coords.toSeq).toSet should not equal partials(1).map(_.
234         coords.toSeq).toSet
235 }
236
237 // =====
238 // ÉTAT STREAMING DANS LE TEMPS : le point le plus récent et le plus à risque
239 // (double-comptage / perte de masse entre micro-batches successifs)

```

```

232 // =====
233
234 test(
235   "état streaming : l'insertion multi micro-batches conserve exactement la
      masse, " +
236   "sans double-comptage ni perte"
237 ) {
238   // Reproduit exactement la boucle interne de StreamKMPlusPlus.run (
      mergeCoresets par
239   // batch, puis global.insert des représentants), mais avec accès direct à
240   // 'global.numPointsSeen' - ce que le callback 'onModel: KMeansModel => Unit
      ' de 'run'
241   // n'expose pas. C'est le test le plus précis pour la classe de bug visée :
      si un point
242   // était compté deux fois (ou perdu) entre deux micro-batches, '
      numPointsSeen'
243   // décrocherait de la somme cumulée AU MOMENT MÊME où ça se produit - vérifi
      é après
244   // CHAQUE batch, pas seulement à la fin (une compensation entre un batch qui
      perd de la
245   // masse et un autre qui en gagne resterait invisible si on ne vérifiait qu'
      à la fin).
246   val k = 4
247   val m = BucketManager.recommendedM(k)
248   val mix = TestData.gaussianMixture(n = 30000, k = k, d = 2, sigma = 0.5,
      seed = 70L)
249
250   val chunks = mix.points.grouped(10000).toArray
251   chunks.length shouldBe 3
252
253   val global = new BucketManager(m, seed = 71L)
254   var cumulativeExpected = 0.0
255
256   chunks.zipWithIndex.foreach { case (chunk, batchId) =>
257     val rdd = spark.sparkContext.parallelize(chunk, numSlices = 4)
258     val partial = StreamKMPlusPlus.mergeCoresets(rdd, StreamKMParams(k = k, m
      = m, seed = 71L + batchId))
259     partial.foreach(global.insert)
260
261     cumulativeExpected += chunk.length.toDouble
262     withClue(s"après le micro-batch $batchId : ") {
263       global.numPointsSeen shouldBe cumulativeExpected +- 1e-6
264     }
265   }
266
267   TestData.totalMass(global.extractCoreset()) shouldBe 30000.0 +- 1e-6
268 }
269
270 // IGNORÉ sous Windows sans winutils.exe/HADOOP_HOME : Structured Streaming a
      besoin
271 // d'écrire les métadonnées de checkpoint sur le système de fichiers local, ce
      qui
272 // passe par RawLocalFileSystem.setPermission → Shell.getWinUtilsPath, absent
      de cette
273 // machine (limitation d'environnement Windows connue de Hadoop, sans rapport
      avec le
274 // code de ce projet). Voir CLAUDE.md, décision 13. Fonctionnera tel quel,
      sans aucune
275 // modification, sur Linux/macOS ou dans un conteneur Docker - à réactiver ('
      test' au
276 // lieu d'ignore') dès que l'un de ces environnements est disponible pour
      lancer la
277 // suite.

```

```

278 ignore("run() sur plusieurs micro-batches produit une qualité comparable à un
    traitement en un seul bloc") {
279     // Test d'intégration bout-en-bout de 'run', via un MemoryStream Spark pilot
    é
280     // manuellement (addData + processAllAvailable) pour contrôler précisément
    les
281     // frontières de micro-batch - c'est le seul test qui exerce réellement
282     // 'StreamKMPlusPlus.run' (les autres tests appellent 'mergeCoresets'
    directement).
283     val k = 4
284     val m = BucketManager.recommendedM(k)
285     val mix = TestData.gaussianMixture(n = 30000, k = k, d = 2, sigma = 0.5,
    seed = 72L)
286     val chunks = mix.points.grouped(10000).toArray
287     chunks.length shouldBe 3
288
289     val sparkSession: SparkSession = spark // alias stable : 'spark' est un 'var
    ', inutilisable dans un import
290     implicit val sqlCtx: SQLContext = sparkSession.sqlContext
291     import sparkSession.implicits._
292     val source = MemoryStream[Array[Double]]
293     val df = source.toDF().withColumnRenamed("value", "features")
294
295     val models = scala.collection.mutable.ArrayBuffer.empty[KMeansModel]
296     val query = StreamKMPlusPlus.run(df, StreamKMParams(k = k, m = m, seed = 73
    L), models += _)
297
298     try {
299         chunks.foreach { chunk =>
300             source.addData(chunk.map(_._coords))
301             query.processAllAvailable()
302         }
303     } finally {
304         query.stop()
305     }
306
307     // Un micro-batch par appel à addData+processAllAvailable : exactement 3 mod
    èles.
308     // Si un batch avait été sauté (perte de masse) ou rejoué (double-comptage),
    ce
309     // nombre resterait 3 mais la qualité ci-dessous décrocherait fortement.
310     models should have length 3
311
312     val streamCost = KMeans.cost(mix.points, models.last.centers)
313     TestData.maxCenterDeviation(mix.trueCenters, models.last.centers) should be
    < 2.0
314
315     // Référence : les mêmes points, en un seul bloc (pas de multi-batch), même
    graine
316     // de driver que le test de masse ci-dessus pour rester comparable.
317     val refBm = new BucketManager(m, seed = 71L)
318     val rdd = spark.sparkContext.parallelize(mix.points, numSlices = 4)
319     StreamKMPlusPlus.mergeCoresets(rdd, StreamKMParams(k = k, m = m, seed = 71L)
    ).foreach(refBm.insert)
320     val refModel = KMeans.fit(refBm.extractCoreset(), k, nRestarts = 5, seed =
    73L)
321     val refCost = KMeans.cost(mix.points, refModel.centers)
322
323     // Tolérance : réutilise le même modèle de composition que le test principal
324     // (CoresetErrorModel), en reconnaissant explicitement qu'il est réemployé
    par
325     // commodité plutôt que re-dérivé pour cette composition précise (batch ->
326     // global.insert, différente du treeReduce interne à chaque batch) - voir

```

```

327 // CLAUDE.md, décision 12.
328 val tolerance = CoresetErrorModel.treeReduceTolerance(numPartitions = 4,
329 depth = 2)
329 withClue(s"streaming (3 micro-batches)=$streamCost, référence (1 bloc)=
330 $refCost : ") {
331     streamCost should be <= refCost * (1.0 + tolerance)
332 }
333 }

```

Listing A.13 – spark/StreamKMSpec.scala

```

1 package streamkm.spark
2
3 import org.apache.spark.sql.SparkSession
4 import org.scalatest.BeforeAndAfterAll
5 import org.scalatest.funSuite.AnyFunSuite
6 import org.scalatest.matchers.should.Matchers
7
8 import streamkm.TestData
9 import streamkm.core.KMeans
10
11 /**
12  * Critères de sortie de l'étape E5.
13  *
14  * Contrairement à 'StreamKMSpec' (qui compare des coresets construits avec de l
15  * 'aléa,
16  * donc avec tolérance), le coût est une simple somme : le test central ici est
17  * une
18  * égalité STRICTE (à la précision flottante près) entre le calcul distribué et
19  * le calcul
20  * séquentiel de référence ('KMeans.cost', déjà validé par 'KMeansSpec'). Un é
21  * cart
22  * signifierait un vrai bug de comptage (points oubliés ou comptés deux fois
23  * lors du
24  * partitionnement), pas une variance d'échantillonnage.
25  */
26 class CostEvaluatorSpec extends AnyFunSuite with Matchers with BeforeAndAfterAll
27 {
28     @transient private var spark: SparkSession = _
29
30     override def beforeAll(): Unit = {
31         spark = SparkSession.builder()
32             .appName("CostEvaluatorSpec")
33             .master("local[4]")
34             .config("spark.ui.enabled", "false")
35             .getOrCreate()
36     }
37
38     override def afterAll(): Unit = {
39         if (spark != null) spark.stop()
40     }
41
42     test("cost : égal (à la précision flottante près) au calcul séquentiel KMeans.
43         cost") {
44         val mix      = TestData.gaussianMixture(n = 12000, k = 4, d = 3, seed = 80L)
45         val model    = KMeans.fit(mix.points, k = 4, nRestarts = 3, seed = 81L)
46         val refCost  = KMeans.cost(mix.points, model.centers)
47
48         val rdd = spark.sparkContext.parallelize(mix.points, numSlices = 6)
49         val distCost = CostEvaluator.cost(rdd, model.centers)
50     }
51 }

```

```

45 // Tolérance relative très fine : seule source d'écart possible = ordre de
46 // flottant entre partitions, pas un biais d'échantillonnage (cf.
47 // CoresetErrorModel,
48 // sans objet ici puisqu'aucun coreset n'est construit par CostEvaluator).
49 distCost shouldBe refCost +- (refCost.abs * 1e-9 + 1e-9)
50 }
51
52 test("cost : invariant au nombre de partitions") {
53   val mix = TestData.gaussianMixture(n = 9000, k = 3, d = 2, seed = 82L)
54   val model = KMeans.fit(mix.points, k = 3, nRestarts = 3, seed = 83L)
55
56   val costs = Seq(1, 3, 9).map { nParts =>
57     val rdd = spark.sparkContext.parallelize(mix.points, numSlices = nParts)
58     nParts -> CostEvaluator.cost(rdd, model.centers)
59   }
60
61   val values = costs.map(_._2)
62   withClue(s"coûts par partitionnement : $costs : ") {
63     values.max - values.min should be <= (values.max.abs * 1e-9 + 1e-9)
64   }
65 }
66
67 test("cost : détecte une régression grossière (mauvais centres = coût très sup
68 // érieur)") {
69   // Garde-fou : vérifie que la fonction n'est pas trivialement toujours d'
70   // accord avec
71   // elle-même (un test qui ne peut jamais échouer ne teste rien). Des centres
72   // tirés
73   // loin de la vraie structure doivent produire un coût nettement plus élevé.
74   val mix = TestData.gaussianMixture(n = 6000, k = 4, d = 2, sigma = 0.5, seed
75   = 84L)
76   val goodModel = KMeans.fit(mix.points, k = 4, nRestarts = 5, seed = 85L)
77
78   val badCenters = Array.fill(4)(Array(1000.0, 1000.0)) // loin de toutes les
79   // gaussiennes
80
81   val rdd = spark.sparkContext.parallelize(mix.points, numSlices = 4)
82   val goodCost = CostEvaluator.cost(rdd, goodModel.centers)
83   val badCost = CostEvaluator.cost(rdd, badCenters)
84
85   badCost should be > goodCost * 10.0
86 }
87
88 test("cost : accepte directement un KMeansModel (surcharge de confort)") {
89   val mix = TestData.gaussianMixture(n = 3000, k = 2, d = 2, seed = 86L)
90   val model = KMeans.fit(mix.points, k = 2, nRestarts = 3, seed = 87L)
91   val rdd = spark.sparkContext.parallelize(mix.points, numSlices = 3)
92
93   CostEvaluator.cost(rdd, model) shouldBe CostEvaluator.cost(rdd, model.
94   centers)
95 }
96 }

```

Listing A.14 – spark/CostEvaluatorSpec.scala

Bibliographie

- [1] Theodoros Bitsakis. *Clustering Big Data Streams in Apache Flink*. Diploma thesis, 2018.
- [2] Marcel R. Ackermann, Marcus Mörtens, Christiane Raupach, Kamil Swierkot, Christiane Lammersen et Christian Sohler. *StreamKM++ : A clustering algorithm for data streams*. ACM Journal of Experimental Algorithmics (JEA), 17, 2012.
- [3] David Arthur et Sergei Vassilvitskii. *k-means++ : The Advantages of Careful Seeding*. Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms, 2007.